

Does Your Reasoning Model Implicitly Know When to Stop Thinking?

Zixuan Huang^{1,2} Xin Xia² Yuxi Ren² Jianbin Zheng² Xuanda Wang² Zhixia Zhang^{1*} Hongyan Xie¹
Songshi Liang² Zehao Chen¹ Xuefeng Xiao² Fuzhen Zhuang³ Jianxin Li¹ Deqing Wang¹ Yikun Ban^{1†}

Abstract

Recent advancements in large reasoning models (LRMs) have greatly improved their capabilities on complex reasoning tasks through long Chains of Thought (CoTs). However, this approach often results in substantial redundancy, impairing computational efficiency and causing significant delays in real-time applications. Recent studies show that longer reasoning chains are frequently uncorrelated with correctness and can even be detrimental to accuracy. In a further in-depth analysis of this phenomenon, we surprisingly uncover and empirically verify that LRMs implicitly know the appropriate time to stop thinking, while this capability is obscured by current sampling paradigms. Motivated by this, we introduce **SAGE** (Self-Aware Guided Efficient Reasoning), a novel sampling paradigm that unleashes this efficient reasoning potential. Furthermore, integrating **SAGE** as mixed sampling into group-based reinforcement learning (**SAGE-RL**) effectively incorporates SAGE-discovered efficient reasoning patterns into standard pass@1 inference, markedly enhancing both the reasoning accuracy and efficiency of LRMs across multiple challenging mathematical benchmarks.

1. Introduction

Reinforcement learning from verifiable rewards (RLVR) algorithms, such as GRPO (Shao et al., 2024; Yang et al., 2026) and GSPO (Zheng et al., 2025a; Lu et al., 2026), have played a pivotal role in enabling test-time scaling. This capability allows large reasoning models (LRMs) like o3 (OpenAI, 2025a) and DeepSeek-R1 (Guo et al., 2025) to

For implement details, contact huang_zx@buaa.edu.cn.
*Second Student Author. ¹Beihang University ²Bytedance China
³School of Artificial Intelligence, Beihang University. Correspondence to: Yikun Ban <yikunb@buaa.edu.cn>.

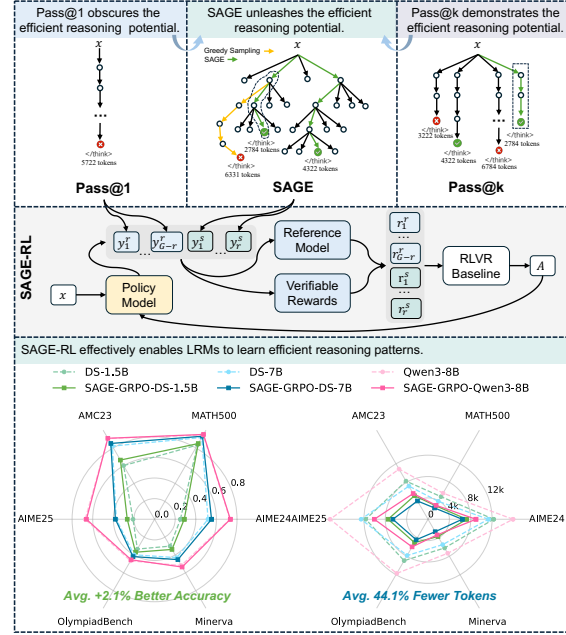


Figure 1. SAGE unleashes the efficient reasoning potential of LRMs obscured by pass@1 and identifies the optimal completions within the model’s capability hidden in pass@k. By enabling LRMs to learn these efficient reasoning patterns, SAGE-RL-tuned models simultaneously enhance reasoning capacity and conciseness on multiple challenging mathematical benchmarks.

“think longer”. Longer CoTs enable LRMs to explore intermediate steps in greater depth and reduce abrupt logical leaps, thereby achieving unprecedented performance on challenging reasoning benchmarks such as AIME (Art of Problem Solving, 2024), OlympiadBench (Chaoqun et al., 2024) and IMO (Luong et al., 2025).

While longer reasoning chains are expected for solving harder problems, prior work shows that length inflation can be uncorrelated with correctness, and that *shorter* chains may in fact yield *better* accuracy. For example, Balachandran et al. (2025) observe that on AIME 2025, DeepSeek-R1 produces responses nearly 5× longer than Claude 3.7 Sonnet while achieving comparable accuracy; Hassid et al. (2025) show that on AIME and HMMT, the shortest responses from QwQ-32B outperform randomly sampled ones by 2 percentage points using 31% fewer tokens. These findings collectively reveal that current CoT outputs often contain

substantial redundancy and irrelevant tokens that do not contribute to the final solution. These unnecessary tokens dramatically reduce reasoning efficiency. This naturally raises a pertinent question: *do LRMs know the appropriate time to terminate thinking?*

We find that, during the exploration of multiple reasoning chains, LRMs consistently assign high confidence to concise yet effective reasoning paths. However, current sampling-based inference strategies typically overlook or fail to select these short and effective chains. Moreover, this phenomenon exhibits clear convergence behavior and becomes increasingly pronounced as the exploration space expands. Taken together, these results strongly indicate that **reasoning models implicitly know the appropriate moment to terminate their reasoning process, but this capability is obscured by current pass@1 training and inference paradigms.**

Motivated by this insight, we introduce **SAGE** (Self-Aware Guided Efficient Reasoning), a simple yet effective decoding strategy that leverages the reasoning model’s self-confidence to discover relatively precise reasoning chains. By incorporating **SAGE** as mixed sampling into group-based reinforcement learning (**SAGE-RL**), we enable the reasoning model to learn concise yet effective thinking patterns without altering its original reasoning paradigm.

In summary, our contributions in this work are as follows:

- We uncover and demonstrate that **LRMs implicitly know the appropriate time to stop thinking**, but this capability is obscured by current sampling paradigms.
- We propose **SAGE**, a novel sampling paradigm that unleashes the efficient reasoning potential of LRMs, simultaneously improving both accuracy and conciseness of reasoning chains.
- We propose **SAGE-RL**, a simple modification to RLVR frameworks that integrates SAGE into the rollout process. As shown in Figure 1, SAGE-RL-tuned models achieve consistent gains across six challenging reasoning benchmarks, including MATH-500, AIME 2024, AIME 2025, AMC23, OlympiadBench and Minerva.

2. Dilemmas of Reasoning Models under Current Sampling Paradigms

To investigate whether reasoning models possess the ability to recognize the appropriate moment to terminate thinking, we first need to re-examine the dilemmas faced by these models under current sampling paradigms.

Pass@k: Scaling CoT length does not lead to correct answers. Assuming that LRMs using current sampling paradigms can reliably stop thinking at the appropriate moment, longer CoTs should outperform shorter ones in lead-

ing to correct solutions. However, extensive experiments involving multiple samplings of the same problem refute this assumption. Balachandran et al. (2025) observes that on AIME 2025, DeepSeek-R1 produces responses nearly 5× longer than Claude 3.7 Sonnet while achieving comparable accuracy; Hassid et al. (2025) also shows that on AIME and HMMT, the shortest responses from QwQ-32B outperform randomly sampled ones by 2 percentage points using 31% fewer tokens. Shrivastava et al. (2025) found that, on AIME 2025, in 72% of problems where both correct and incorrect answers were generated, the longer response was more likely to be incorrect than the shorter one.

These findings collectively reveal that: once the chain-of-thought length reaches a certain threshold, simply scaling the length further does not lead to a corresponding improvement in the model’s reasoning capability. *Furthermore, the optimal response within the model’s capability is obscured by existing sampling paradigms and can currently only be retrieved post hoc through test-time scaling methods.*

Pass@1: Existing sampling strategies fail to enable timely termination of thinking. To gain a finer-grained understanding of these findings and precisely locate its root cause, we build upon this observation and take a step further. Reasoning tasks, particularly in mathematical reasoning and code generation, typically require step-by-step answers. Leveraging this observation, we introduce a simple metric to quantify the efficient reasoning capability of models: the **Ratio of the First Correct Step** (RFCS), defined as the step index at which the correct answer first appears divided by the total number of reasoning steps.

Specifically, we utilize DeepSeek-distilled-Qwen-1.5B (DS-1.5B), (DeepSeek-AI, 2025), DeepScaleR (Luo et al., 2025b) and Qwen3-8B (Yang et al., 2025a) to generate answers for MATH-500 (Lightman et al., 2023) problems. For each response, we segment it into distinct reasoning steps by “\n\n” (Chen et al., 2025a) and compute RFCS for each problem. As illustrated in Figure 2, the model correctly derives the answer using only 500 tokens, yet under the current sampling strategy, it continues with an additional 452 redundant tokens before terminating the reasoning process. This clearly demonstrates that the LRM fails to end its thinking at the appropriate moment on this problem.

Such cases are not isolated in our study. More statistical results are summarized in Figure 3, where **RFCS(< 1)** and **RFCS(avg)** respectively denote the number of correct responses where RFCS is not equal to 1 and the average RFCS value across all correct responses. From the statistical results, all models exhibit significant ineffective steps in over half of the samples. Moreover, compared to DS-1.5B, models with higher post-training extent (DeepScaleR), or more advanced reasoning capabilities (Qwen3-8B) show no substantial improvement on this metric. This indicates that,

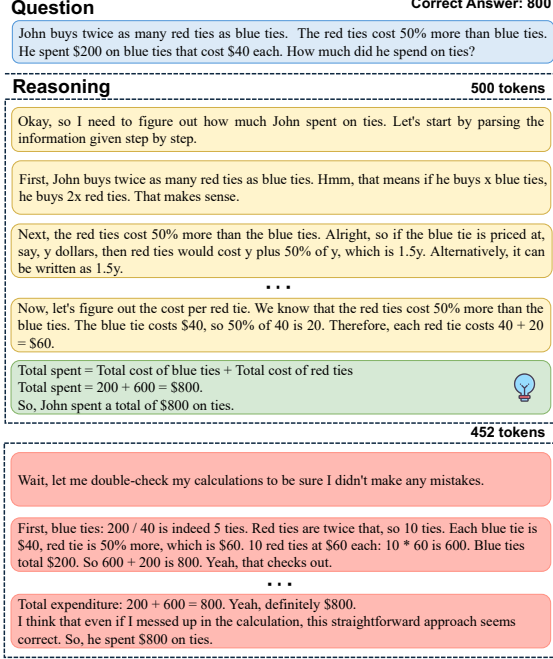


Figure 2. Illustration of the step-by-step answering process.

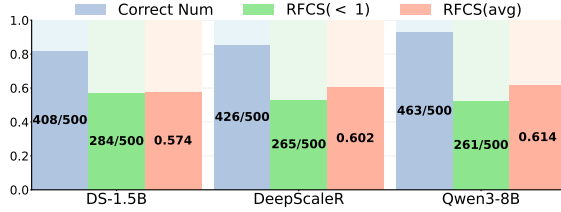


Figure 3. Statistics of RFCS on MATH500 across LLMs.

in general scenarios, existing reasoning models struggle to terminate their thinking process at the appropriate moment under the current inference paradigm (i.e., pass@1).

In summary, the surprising performance of relatively shorter responses in pass@k reveals the inherent potential of the model for efficient reasoning. The pervasive redundancy of reasoning steps in pass@1 indicates that current sampling paradigms obscure this potential.

Therefore, we attempt to adopt a sampling strategy with a larger exploration space built upon pass@1 to intentionally uncover the precise reasoning chains that are hidden within the broader pass@k distribution.

3. Intentionally Exploring Shorter CoTs

Notations. Given a query $\mathbf{x}$ and a prefix $\mathbf{y}_{<k} = (y_1, y_2, \dots, y_{k-1})$ previously generated by the language model π_θ , We define Φ as the average cumulative log-probability up to generation step k , where $\phi(y_i; \mathbf{y}_{<i})$ is

the (next-token) log-probability of the i -th token in π_θ :

$$\Phi(\mathbf{y}_{\leq k}) = \frac{1}{k} \sum_{i=1}^k \phi(y_i; \mathbf{y}_{<i}). \quad (1)$$

$$\phi(y_i; \mathbf{y}_{<i}) = \log \pi_\theta(y_i | \mathbf{y}_{<i}, \mathbf{x}). \quad (2)$$

Token-Wise Reasoning Path Exploration. We first propose a token-wise reasoning path expansion algorithm until a maximum step budget $T_{\max}$ is reached. With exploration width m (denoted as **EW**), we maintain the top- m candidate sequences according to the scoring function Φ , and expands them in subsequent decoding steps (Meister et al., 2020).

Formally, given a set of m candidate sequences $Y_{i-1} = \{\mathbf{y}_{\leq i-1}^{(1)}, \mathbf{y}_{\leq i-1}^{(2)}, \dots, \mathbf{y}_{\leq i-1}^{(m)}\}$ at timestep $i-1$, for $\mathbf{y}_{\leq i-1}^{(j)} \in Y_{i-1}, j \in [1, m]$, we select the top $2m$ most probable tokens

$$\mathcal{T}^{(j)} = \text{Top}_{2m}(\{y_i | y_i \in \mathcal{V}; \phi(\cdot; \mathbf{y}_{\leq i-1}^{(j)})\}) \quad (3)$$

where $\text{Top}_m(\cdot; \phi)$ denotes an operator that ranks all candidate elements in descending order according to their ϕ -scores and returns the subset consisting of the top $2m$ elements with the highest scores. This yields a candidate group of size $2m \times m = 2m^2$, formally written as

$$\hat{Y}_i = \{\mathbf{y}_{\leq i}^{(j,k)} | j \in [m], k \in [2m]\}, \quad (4)$$

where each candidate sequence is constructed by appending the k -th best token to the j -th beam:

$$\mathbf{y}_{\leq i}^{(j,k)} = \mathbf{y}_{\leq i-1}^{(j)} \oplus y_i^{(j,k)}, \quad y_i^{(j,k)} \in \mathcal{T}^{(j)}. \quad (5)$$

We retain the top- m highest-scoring candidate sequences for next iteration:

$$Y_i = \text{Top}_m(\{\mathbf{y}_{\leq i}^{(j,k)} | j \in [m], k \in [2m]\}; \Phi). \quad (6)$$

Exploration Termination. We denote the Tolerance accept rank Ratio of $\langle \text{think} \rangle$ $h/2m$ as **TR**, where $h \in \{1, 2, \dots, 2m\}$ is a hyperparameter representing the tolerance for the rank of $\langle \text{think} \rangle$. Given the required number of CoTs $r \in \{1, 2, \dots, m\}$, once we have reached a candidate sequence $\mathbf{y}_{\leq i}^{(j,k)}$, where $y_i^{(j,k)}$ is $\langle \text{think} \rangle$ and within the top- h probable tokens $\text{top}_h \mathcal{T}^{(j)}$, we add it as a completion to the candidate sequence set $\mathcal{O}$. Otherwise, we discard this candidate sequence, as the model’s confidence in terminating the thinking process is low at this point (Liu et al., 2025). When $|\mathcal{O}| \geq r$, we terminate the entire process. If when $i = T_{\max}$ and $|\mathcal{O}| < r$, we also add $\text{top}_{r-|\mathcal{O}|}(\{\mathbf{y}_{T_{\max}} \oplus \langle \text{think} \rangle\}; \Phi)$ to $\mathcal{O}$ to ensure $|\mathcal{O}| = r$.

Greedy Sampling of the Answers. Through the above process, we generate r reasoning chains $\mathbf{t}_i \in \mathcal{O}$ for each question $\mathbf{x}$. Next, we derive the answer $\pi_\theta(\mathbf{a}_i | \mathbf{x}, \mathbf{t}_i)$ greedily based on the query and the internal reasoning chains. Ultimately, for each question $\mathbf{x}$, our decoding strategy generates r completions $\{\mathbf{t}_i, \mathbf{a}_i\}, i \in [r]$.

Notably, although our algorithm is built upon vanilla beam search, it exhibits significant differences. We provide a detailed comparative analysis in Appendix B.

4. Your Reasoning Model Implicitly Knows When to Stop Thinking

Built upon the observations of Section 2 and take a step further, we conduct analytical experiments involving the following related algorithms:

TSearch($\mathbf{m}, \mathbf{r}$) w/ Φ denotes the algorithm from Section 3 with exploration width m and r returned completions.

TSearch($\mathbf{m}, \mathbf{r}$) w/ ϕ is a TSearch variant used to ablate the role of Φ , which greedily retains the top- m candidate sequences with the most probable new token at each step according to the following equation instead of Equation 6:

$$Y_i = \left\{ \mathbf{y}_{\leq i}^{(j,k)} \mid (j,k) \in \arg \text{Top}_m \left(\{ \mathbf{y}_i^{(j,k)} \mid j \in [m], k \in [2m] \}; \phi(\cdot; \mathbf{y}_{\leq i-1}^{(j)}) \right) \right\}. \quad (7)$$

EW = 0 denotes greedy sampling, which essentially represents a degeneration of TSearch with no exploration. Li et al. (2023) found that its performance is comparable to the average results obtained from random sampling.

Random refers to standard random sampling with temperature and top-p both set to 1.0.

4.1. High-Confidence Paths Lead to Efficient Reasoning

We use reasoning chains retained by Φ to represent the high-confidence paths generated during TSearch. To assess the role of the Φ during this process, we compare TSearch w/ Φ with TSearch w/ ϕ across increasing exploration width m .

Enlarging the exploration width m influences TSearch in two contrasting ways. On the positive side, a broader candidate token window $\mathcal{T}$ facilitates the discovery of more varied reasoning paths and improves the probability of identifying optimal solutions among pass@k samples (Shrivastava et al., 2025; Hassid et al., 2025). On the negative side, a larger $\mathcal{T}$ is used for $\langle \text{think} \rangle$ detection. With $\text{TR} = 1$, termination occurs immediately upon $\langle \text{think} \rangle$ ’s appearance, probably leading to significant length collapse. The results in Figure 4 clearly demonstrate the pivotal role of Φ .

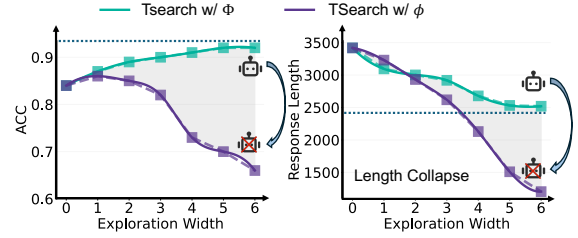


Figure 4. Comparison of TSearch variants with increasing EW on DS-7B and a randomly selected subset of MATH-500 (size = 100) under a 10k token budget. To directly investigate the influence of Φ , we uniformly set $\text{TR} = 1$.

Observation 1 (Figure 4). In TSearch w/ Φ , increasing m leads to a consistent reduction in response length accompanied by a steady improvement in accuracy. By contrast, TSearch w/ ϕ suffers a rapid degradation in accuracy that closely tracks the sharp decline in response length. Furthermore, enlarging the exploration space represents an opportunity to enhance reasoning chain quality when Φ is present, whereas its absence makes length collapse and performance deterioration an inevitable consequence. These results indicate that **the high-confidence branches preserved by Φ are not only markedly shorter, but also substantially more effective.**

4.2. High-Confidence Paths Lead to Confident Ends

To further investigate the length collapse problem in TSearch w/ ϕ illustrated in Section 4.1, we apply TR to drop branches concluded with low confidence. The experimental results are shown in Table 1.

Table 1. Comparison of TSearch (4,1) variants under different TR with the same settings of Figure 4. When $\text{TR} < 1$, TSearch prunes candidate sequences where the rank ratio of $\langle \text{think} \rangle$ within $\mathcal{T}$ is lower than TR . **ACC** denotes the accuracy, **LEN** refers to the average response length, **T-LEN** represents the average number of think tokens.

Method	TR	ACC	T-LEN	LEN
Random	-	0.84	3126	3419
TSearch (4,1) w/ ϕ	1.00	0.79	1712	2129
TSearch (4,1) w/ ϕ	0.75	0.82	2022	2333
TSearch (4,1) w/ ϕ	0.50	0.89	2176	2609
TSearch (4,1) w/ Φ	1.00	0.92	2213	2609
TSearch (4,1) w/ Φ	0.75	0.92	2221	2621
TSearch (4,1) w/ Φ	0.50	0.91	2212	2632

As for TSearch w/ Φ , varying the TR has virtually no impact on performance. By comparison, it exerts a strong influence on TSearch w/ ϕ . This indicates a strong correlation between the presence of Φ and the ranking of $\langle \text{think} \rangle$ within $\mathcal{T}$. To further study the correlation between them, we record

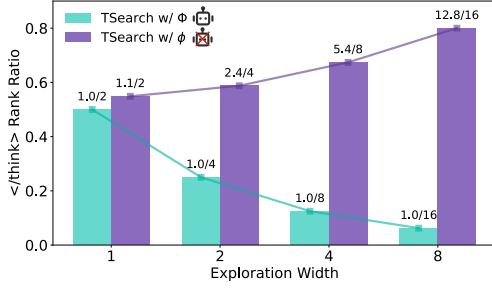


Figure 5. The average rank ratio of $\langle \text{think} \rangle$ in $\mathcal{T}$ upon appearance.

the average rank ratio at which $\langle \text{think} \rangle$ appears during TSearch and illustrate them in Figure 5.

We observe that as **EW** increases, the $\langle \text{think} \rangle$ token identified by TSearch w/ Φ consistently ranks **first** within the candidate set $\mathcal{T}$ at the moment it appears when evaluated by Φ . This behavior indicates that the policy is highly confident in terminating the reasoning process once $\langle \text{think} \rangle$ enters $\mathcal{T}$. In contrast, for TSearch w/ ϕ , the rank ratio of the $\langle \text{think} \rangle$ token gradually increases as measured by ϕ , suggesting increasing uncertainty about whether the next token should be $\langle \text{think} \rangle$. This discrepancy explains the significant differences in the role of **TR** between TSearch w/ Φ and TSearch w/ ϕ , as reported in Table 1.

Observation 2 (Figure 6). The policy implicitly exhibits high confidence in terminating a high-confidence reasoning chain, as supported by TSEARCH with the cumulative probability Φ . However, the final $\langle \text{think} \rangle$ token may have a relatively low next-token probability, which is revealed by TSEARCH w/ ϕ . This discrepancy indicates that many short yet high-quality reasoning chains are likely to be overlooked by greedy or random sampling strategies.

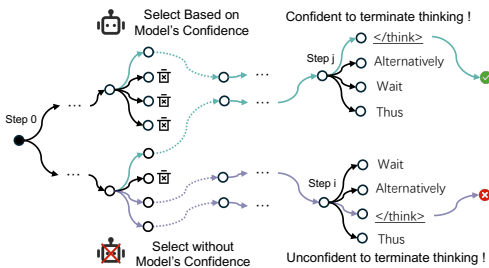


Figure 6. Illustration of Observation 2. When reasoning branches are retained according to the model’s confidence at each expansion step, the model is able to conclude them with strong confidence.

4.3. Scaling Exploration Drives Capability Convergence

In this section, we conduct further experiments to probe the upper boundary of the efficient reasoning capability illustrated in Section 4.1. Specifically, under sufficient token

budget $T_{\max} = 32,768$, we adopt TSearch ($m, 1$) w/ Φ as the sampling strategy, and compare the pass@1 and response length of DS-1.5B and DeepScaleR on MATH-500 and AMC23 as the exploration width m increases. An increase in m corresponds to a larger exploration space during the generation of reasoning chains. The results are shown in Figure 18. For a clearer visualization of the model’s performance trends, we measure reasoning efficiency for each run in Figure 18 using token efficiency (pass@1 / response length), as illustrated in Figure 7.

(1) At **EW** = 0, the model operates in a completely non-exploratory regime and exhibits limited reasoning efficiency. This indicates that standard non-exploratory greedy or random sampling constrains the model’s inherent ability, which is fully consistent with the observations in Section 2.

(2) As shown in Figure 18, enlarging the exploration width leads to consistent improvements in pass@1 while simultaneously reducing response length, with both metrics exhibiting a trend toward gradual convergence. This trend further verifies reasoning models’ inherent efficient reasoning capability. From Figure 7, we can clearly find that this capability is progressively unleashed as the exploration width grows.

(3) LRMs gradually approach the boundary of their inherent efficient reasoning capability as the degree of exploration increases, and this phenomenon is not an isolated occurrence but a universal pattern observed across models and datasets.

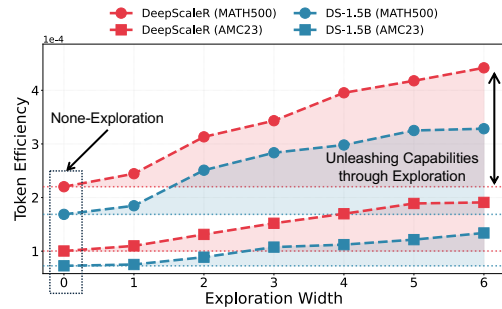


Figure 7. Token efficiency comparison on each run in Figure 18.

Observation 3 (Figure 7). As the exploration space expands during reasoning, LRM is increasingly capable of identifying precise and compact reasoning paths with high confidence. Furthermore, with the continued growth of the exploration space, this behavior demonstrates an obvious convergence trend.

Furthermore, as a post-trained version of DS-1.5B, DeepScaleR exhibits steeper token efficiency improvement on both MATH-500 and AMC23. This suggests that greater post-training enhances the model’s ability to leverage increased exploration space for unleashing its intrinsic efficient reasoning potential.

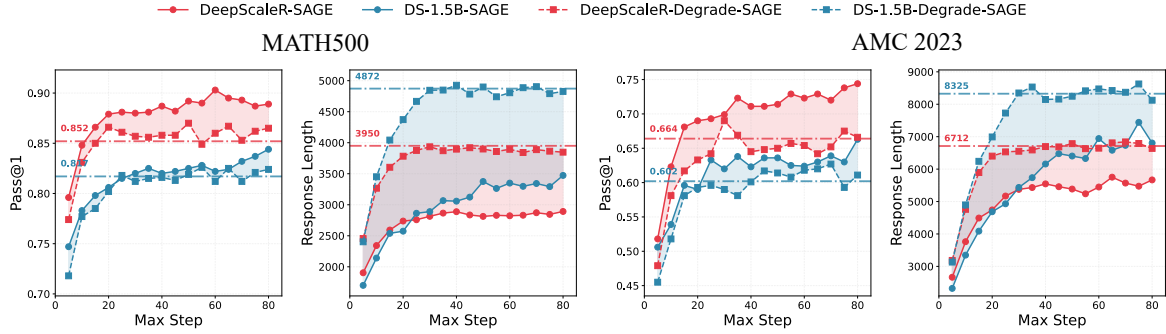


Figure 8. Performance comparison with SAGE and Degrade-SAGE on MATH-500 and AMC23 under different generation step budgets.

In summary, when provided with adequate exploration space, LRMs can identify precise and concise reasoning chains with high confidence and appropriately terminate the reasoning process, indicating that these models possess an inherent sense of when to stop reasoning. By contrast, current purely sampling-based strategies implicitly limit this capability of LRMs by relying solely on the next-token probability distribution.

5. Self-Aware Guided Efficient Reasoning

5.1. Methodology

While Section 4.3 demonstrates that TSearch w/ Φ effectively unleashes the efficient reasoning potential of LRMs as the exploration space expands, the method remains inherently **greedy**. Our goal, however, is to translate this insight into **random sampling**-based inference paradigms. Fortunately, from prior analysis in Section 4.2, when Φ is present, $\langle \text{think} \rangle$ consistently achieves the top rank upon appearance. This observation implies that TSearch w/ Φ is effectively equivalent to directly identifying reasoning steps that terminate with $\langle \text{think} \rangle$, rendering token-level reasoning chain expansion unnecessary. Based on this observation and built upon TSearch w/ Φ , we introduce **Self-Aware Guided Efficient Reasoning (SAGE)**, a simple yet effective sampling paradigm that performs *step-wise* reasoning chain expansion, with a more detailed motivation of this token-to-step transition in Appendix C. SAGE differs from TSearch w/ Φ in only the following two respects:

Step-Wise Reasoning Chain Exploration. At step i , each candidate sequence is extended by one full reasoning step $\mathbf{r}$ until the maximum reasoning step limit $T_{\max}$ is reached:

$$\mathbf{y}_{\leq i}^{(j,k)} = \mathbf{y}_{\leq i-1}^{(j)} \oplus \mathbf{r}_i^{(j,k)}, \quad \mathbf{r}_i^{(j,k)} \in \mathcal{R}^{(j)}, \quad (8)$$

where $\mathcal{R}^{(j)} \triangleq \{\mathbf{r}_i^{(j,1)}, \mathbf{r}_i^{(j,2)}, \dots, \mathbf{r}_i^{(j,2m)}\}$ denotes the set of $2m$ reasoning steps independently sampled from the policy π_θ conditioned on the query $\mathbf{x}$ and prefix $\mathbf{y}_{\leq i-1}^{(j)}$ using vanilla random sampling. This process replaces the token-level

expansion in Equation 5.

Exploration Termination. Based on the conclusions in Section 4.2, we no longer need to manually set the tolerance rank ratio $\mathbf{TR}$ as the high-confidence reasoning branches consistently lead to confident ends. Our termination condition can be simply defined as: If we have reached a candidate sequence $\mathbf{y}_{\leq i}^{(j,k)}$, where $\mathbf{r}_k^{(j,k)}$ ends with $\langle \text{think} \rangle$, we add it as a completion to the candidate sequence set $\mathcal{O}$.

5.2. SAGE Inference Scaling Trends with Step Budget

We introduce a step-wise alternative to random sampling namely Degrade SAGE to ablate the exploration space of SAGE. Degrade SAGE directly samples one reasoning step at each iteration until $\langle \text{think} \rangle$ appears or $T_{\max}$ is reached. To balance computational efficiency and performance (discussed in Appendix E.4), we adopt SAGE (2,1) as the representative of our algorithm. We scale the maximum reasoning step budget gradually and compare the pass@1 and response length of SAGE and Degraded SAGE on MATH-500 (mean@4) and AMC23 (mean@16) respectively. We mark *Random* results for DeepScaleR and DS-1.5B at 32,768 token budget with red and blue dashed lines, respectively.

(1) The inference scaling trends of SAGE demonstrate the model’s capability to terminate thinking at appropriate timings. Under constrained step budgets, SAGE outperforms Degraded SAGE in pass@1 with similar sequence lengths. This advantage stems from SAGE stopping thinking earlier, leading to more complete CoTs. When step budgets are ample, a relatively stable performance gap emerges between SAGE and Degraded SAGE. Here, with token count no longer a bottleneck, the difference stems solely from reasoning chain exploration. These results clearly show that *SAGE effectively identifies reasoning chains superior to those of Degraded SAGE, as they are both shorter and more likely to lead to correct answers.*

(2) SAGE prioritizes performance for strong models and hard datasets, and efficiency for weaker models and simple datasets. On stronger DeepScale and harder AMC23,

Does Your Reasoning Model Implicitly Know When to Stop Thinking?

Table 2. Pass@1, response length (LEN) and token efficiency (TE) results on four complex mathematical benchmarks. TE is calculated as Pass@1 / LEN. **Bold** and underlined denote the best and second-best results.

Method	MATH-500			AIME 2024			AIME 2025			OlympiadBench		
	Pass@1(%)	LEN1	TE↑(×10 ⁻³)	Pass@1(%)	LEN1	TE↑(×10 ⁻³)	Pass@1(%)	LEN1	TE↑(×10 ⁻³)	Pass@1(%)	LEN1	TE↑(×10 ⁻³)
DS-1.5B	83.2	4882	17.0	25.1	12300	2.04	20.9	11669	1.79	33.4	8954	3.73
+ LC-R1	80.4 (12.8)	2973 (11909)	27.0 (158.8%)	23.3 (11.8)	7098 (15202)	3.28 (160.8%)	20.9 (10.0)	6942 (14727)	3.01 (168.2%)	32.0 (11.4)	4632 (14322)	6.91 (185.3%)
+ ThinkPrune-2k	81.7 (11.5)	2826 (12056)	28.9 (170.0%)	23.7 (11.4)	<u>7085</u> (15215)	3.35 (164.2%)	19.7 (11.2)	6918 (14751)	2.85 (159.2%)	32.9 (10.5)	4752 (14202)	6.92 (185.5%)
+ AdaptThink	80.4 (12.8)	2563 (12319)	31.4 (184.1%)	25.7 (10.6)	8055 (14245)	3.19 (156.4%)	21.8 (10.9)	8155 (13514)	2.67 (149.2%)	32.6 (10.8)	4563 (14391)	7.14 (191.4%)
+ Efficient Reasoning	82.0 (11.2)	2821 (12061)	29.1 (170.6%)	26.2 (11.1)	9189 (13111)	2.85 (139.7%)	22.9 (12.0)	8590 (13079)	2.67 (149.2%)	33.8 (10.4)	5755 (13199)	5.87 (157.4%)
+ GRPO	83.6 (10.4)	3907 (1975)	21.4 (125.6%)	28.3 (13.2)	8767 (13533)	3.23 (158.3%)	24.1 (13.2)	8263 (13406)	2.92 (163.1%)	34.2 (10.8)	6323 (12631)	5.41 (145.0%)
+ SAGE-GRPO	<u>84.8</u> (11.6)	2915 (11967)	29.1 (170.7%)	28.8 (13.7)	7243 (15057)	<u>3.98</u> (195.1%)	<u>26.5</u> (15.6)	7479 (14190)	<u>3.54</u> (197.8%)	<u>36.9</u> (13.5)	<u>5050</u> (13904)	7.31 (196.0%)
+ GSPO	83.4 (10.2)	3898 (1984)	25.3 (121.4%)	28.3 (13.2)	8604 (13696)	3.29 (161.3%)	25.1 (14.2)	8227 (13442)	3.05 (170.4%)	34.6 (11.2)	6410 (12544)	5.40 (144.8%)
+ SAGE-GSPO	85.2 (12.0)	2921 (11961)	<u>29.2</u> (171.6%)	<u>28.5</u> (13.4)	6889 (15411)	4.14 (1102.9%)	<u>27.1</u> (16.2)	<u>7167</u> (14502)	3.78 (1111.1%)	37.3 (13.9)	5172 (13782)	<u>7.21</u> (193.3%)
DeepScaleR	86.0	3805	22.6	31.4	9370	3.35	25.4	9310	2.73	35.9	5972	6.01
+ ThinkPrune-2k	82.5 (13.5)	2946 (1859)	<u>28.0</u> (123.9%)	33.5 (12.1)	8108 (11262)	4.13 (123.3%)	26.0 (10.6)	7486 (11824)	<u>3.47</u> (127.1%)	35.1 (10.8)	4723 (11249)	<u>7.43</u> (123.6%)
+ GRPO	<u>87.6</u> (11.6)	3482 (1323)	25.2 (111.3%)	<u>35.6</u> (14.2)	<u>8592</u> (1778)	<u>4.14</u> (123.6%)	27.4 (12.0)	8185 (11125)	3.35 (122.7%)	<u>36.2</u> (10.3)	5443 (1529)	6.65 (110.6%)
+ SAGE-GRPO	88.8 (12.8)	<u>3117</u> (1688)	28.4 (125.7%)	36.1 (14.7)	8094 (11276)	4.46 (133.1%)	<u>27.2</u> (11.8)	<u>7704</u> (11606)	3.53 (129.3%)	36.5 (10.6)	<u>4890</u> (11082)	7.46 (124.1%)
DS-7B	91.6	3871	23.7	51.9	11305	4.59	37.1	12540	2.96	39.8	7839	5.08
+ LC-R1	87.3 (14.3)	2076 (11795)	<u>42.1</u> (177.7%)	51.7 (10.2)	6820 (14485)	7.58 (165.1%)	35.7 (11.4)	7458 (15082)	<u>4.79</u> (161.8%)	<u>41.4</u> (11.6)	4193 (13646)	9.87 (194.3%)
+ AdaptThink	88.9 (12.7)	2199 (11672)	40.4 (170.9%)	52.1 (10.2)	6679 (14626)	<u>7.80</u> (169.9%)	35.0 (12.1)	7807 (14733)	4.48 (172.3%)	38.9 (10.9)	4915 (12924)	7.91 (155.7%)
+ Efficient Reasoning	89.8 (11.8)	2408 (11463)	37.3 (157.6%)	51.9 (10.0)	6667 (14638)	7.78 (169.5%)	36.2 (10.9)	7501 (15039)	4.82 (162.8%)	40.1 (10.3)	4599 (13240)	8.72 (171.7%)
+ GRPO-LEAD	89.5 (12.1)	2752 (11119)	32.5 (137.1%)	<u>53.1</u> (11.2)	7023 (14282)	7.56 (164.7%)	36.1 (11.0)	7842 (14698)	4.60 (155.4%)	40.6 (10.8)	4972 (12867)	8.17 (160.8%)
+ GRPO	<u>92.0</u> (10.4)	3219 (1652)	28.5 (120.2%)	52.5 (10.6)	8424 (12881)	6.23 (135.7%)	38.4 (11.3)	10123 (12417)	3.79 (128.0%)	41.2 (11.4)	5498 (12341)	7.50 (147.6%)
+ SAGE-GRPO	93.0 (11.4)	<u>2141</u> (11730)	43.4 (183.1%)	55.3 (13.4)	6422 (14883)	8.61 (187.6%)	<u>38.0</u> (10.9)	6583 (15957)	<u>5.77</u> (194.9%)	41.8 (12.0)	<u>4435</u> (13404)	<u>9.42</u> (185.4%)
Qwen3-8B	94.4	5640	16.7	73.2	15920	4.60	67.3	18342	3.67	46.6	11707	4.00
+ GRPO	93.6 (10.8)	4470 (11170)	20.9 (125.1%)	72.8 (10.4)	10573 (15347)	6.89 (149.8%)	<u>66.6</u> (10.7)	13981 (14361)	4.76 (129.7%)	45.1 (11.5)	7512 (14195)	6.00 (150.0%)
+ SAGE-GRPO	95.0 (10.6)	<u>3015</u> (12625)	<u>31.5</u> (188.2%)	<u>73.5</u> (10.3)	<u>8975</u> (16945)	<u>8.19</u> (178.0%)	<u>66.6</u> (10.7)	<u>10052</u> (18290)	<u>6.58</u> (179.3%)	45.4 (11.2)	<u>5972</u> (15735)	<u>7.60</u> (190.9%)
+ GSPO	94.6 (10.2)	4342 (11298)	22.2 (132.9%)	73.0 (10.2)	10544 (15376)	6.92 (150.4%)	66.2 (11.1)	14082 (14260)	4.70 (130.2%)	46.6 (10.0)	7964 (13743)	5.85 (146.2%)
+ SAGE-GSPO	94.4 (10.0)	2753 (12887)	34.3 (1105.3%)	73.7 (10.5)	8547 (17373)	8.62 (187.4%)	66.0 (11.3)	9183 (19159)	7.19 (195.9%)	46.7 (10.1)	5436 (16271)	8.59 (1114.7%)

we observe greater pass@1 gains. In contrast, on weaker DS-1.5B and simpler MATH-500, we note larger response length reductions. **From the model’s perspective**, stronger models have a higher capability ceiling, enabling SAGE to deliver larger accuracy gains with more necessary tokens. In contrast, weaker models suffer from more severe overthinking, creating more chances for token redundancy reduction. **From the dataset’s perspective**, LRMs can solve most problems on easier datasets, making response length the key optimization goal. By exploiting the model’s inherent sense of when to stop thinking, SAGE identifies shorter reasoning chains to reduce response length significantly. Conversely, harder datasets contain more challenging problems requiring more tokens to solve, and SAGE boosts accuracy notably on them, confirming its efficacy on uncovering correct reasoning chains with minimal necessary tokens.

6. SAGE-RL: Integrating Efficient Reasoning Patterns into Current Inference Paradigms

As shown in Section 5, SAGE effectively unleashes reasoning models’ implicit capacity for efficient reasoning. An appealing extension is to incorporate the efficient reasoning pattern uncovered by SAGE into standard pass@1 inference. Thus, we introduce **SAGE-RL**, a simple modification to RLVR, to achieve this goal.

Given a question q , RLVR typically samples a group of responses $\mathcal{G} = \{o_1, \dots, o_G\}$ from the current policy. The sole difference between SAGE-RL and RLVR lies in the rollout phase, where SAGE-RL employs a hybrid sampling strategy. SAGE-RL employs SAGE (m,r) to generate r

responses $\{o_1^S, o_2^S, \dots, o_r^S\}$ and uses standard random sampling for the remaining $G-r$ responses $\{o_1^R, o_2^R, \dots, o_{G-r}^R\}$. Ultimately, the rollout phase in SAGE-RL yields the set of responses $\mathcal{G} = \{o_1^S, \dots, o_r^S, o_1^R, \dots, o_{G-r}^R\}$ for each q .

7. Experiments

We apply both RLVR (GRPO (Shao et al., 2024), GSPO (Zheng et al., 2025a)) and corresponding SAGE-RL method (SAGE-GRPO, SAGE-GSPO) to tune four widely adopted LRMs with a group size of $G = 8$. The training objectives of these algorithms can be found in Appendix D.1. Within each group, SAGE-RL employs SAGE (2,2) to search for two completions with precise reasoning chains, while the remaining six completions are obtained through default random sampling in ver1. We also compare with existing open-source methods, including LC-R1 (Cheng et al., 2025), ThinkPrune (Hou et al., 2025), AdaptThink (Zhang et al., 2025), Efficient-Reasoning (Arora et al., 2025), and GRPO-LEAD (Zhang et al., 2025). Additional implementation details are provided in Appendix D.2 due to space constraints.

7.1. Main Results

Table 2 presents a performance comparison among SAGE-RL and baselines. Due to space constraints, we present results from only four out of the six evaluated datasets. The complete experimental results and additional analysis are provided in Appendix E.

(1) SAGE-RL achieves comprehensive improvements in both reasoning capability and token efficiency. As

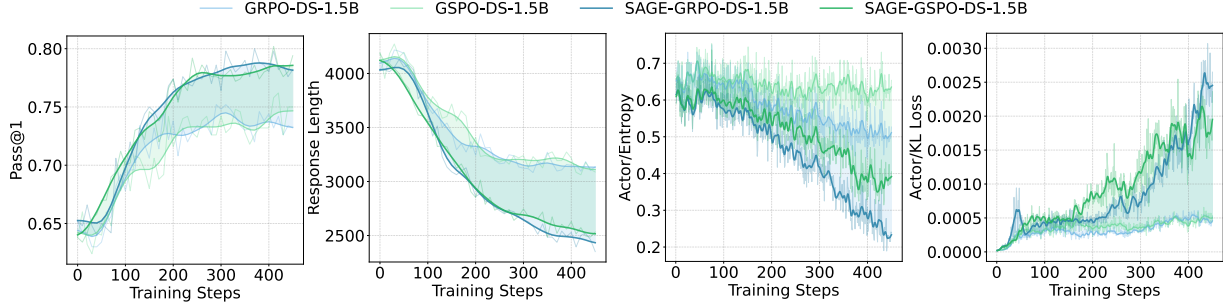


Figure 9. Training Dynamics comparison between RLVR and SAGE-RL. The left two figures present results evaluated every 10 steps on MATH-500 under an 8,192 token budget. The right two figures illustrate the entropy and KL divergence of the policy for every step.

shown in Table 2, most baselines achieve token compression at the cost of reduced reasoning capability. For instance, on MATH-500, AdaptThink compresses the token count of DS-1.5B from 4,882 to 2,563, but at the expense of a 2.8% drop in pass@1. Similar performance degradation is also widely observed across AIME 2024, AIME 2025 and OlympiadBench. RLVR was initially proposed to improve reasoning performance through extended reasoning lengths (DeepSeek-AI, 2025), yet existing baselines compromise this capability to different extents.

In contrast, SAGE-RL consistently achieves the best or second-best token efficiency across all benchmarks, while effectively improving the base models’ capabilities on these complex reasoning tasks. This is because SAGE-RL achieves efficient reasoning by enabling LRMs to learn more precise reasoning chains, simultaneously shortening the inference trajectories while enhancing reasoning capability. As illustrated in Figure 8, the reasoning chains sampled by SAGE are shorter than those from standard sampling and more effectively guide the model toward correct solutions. In group-based comparison processes similar to GRPO, this advantage is amplified by the baseline’s regularization. Since SAGE more frequently yields high-reward outcomes, the policy model naturally shifts its reasoning patterns toward the efficient modes discovered by SAGE.

(2) SAGE-RL effectively enables LRMs to learn efficient reasoning patterns. As shown in Table 2, although vanilla GRPO and GSPO moderately improve the reasoning capability of LRMs compared to other baselines, the inference trajectories learned by LRMs from standard random sampling still contain substantial token redundancy. Consequently, the overall token efficiency remains significantly lower than that of efficient reasoning baselines. In contrast, SAGE-RL achieves substantial improvements in both reasoning capability and token efficiency. Since the only difference lies in the sampling strategy for 2 out of 8 samples per group, the results demonstrate that SAGE-RL effectively enables the policy model to learn shorter yet more accurate reasoning patterns.

Figure 9 clearly illustrates this process. As training progresses, deploying SAGE-RL on both GRPO and GSPO leads to more pronounced improvements in pass@1 and greater reductions in response length. In contrast to standard RLVR, SAGE-RL shows a more significant entropy reduction, suggesting that the policy model gradually acquires the precise reasoning chains identified by SAGE, resulting in greater confidence during inference as training progresses. In terms of KL divergence, SAGE-RL also exhibits a more pronounced increasing trend. This indicates that the policy model deviates more significantly from the original probability distribution as training progresses. Such behavior suggests that the reasoning chains generated by SAGE, compared to those from random sampling, induce larger updates in the model. This is primarily because unleashing the model’s efficient reasoning capability requires more substantial updates to learn reasoning patterns that differ markedly from the original ones.

As SAGE-RL’s improvement solely stems from the rollout phase, the direct comparison with RLVR in this section serves as an effective **ablation study** of our approach.

7.2. Computational Cost Analysis

The additional cost of SAGE-RL stems from replacing 2 of 8 standard rollouts with SAGE(2,2). SAGE(2,2) keeps 4 candidate sequences, introducing 4× GPU memory overhead for these two trajectories. With sufficient parallelism, the wall-clock time is comparable. However, SAGE’s step-by-step inference requires repeated calls to vLLM’s generate operator, reducing the efficiency that standard random sampling enjoys. Moreover, as shown in Figure 9, SAGE-RL drives faster convergence toward shorter outputs, which significantly reduces rollout latency.

We report the overhead results for DS-1.5B. To ablate the impact of vLLM, we also implemented SAGE-GRPO using native HF inference. We compare the training cost of SAGE-GRPO with several baselines. As shown in Table 3, SAGE-GRPO introduces only moderate additional wall-clock cost under ver1, while maintaining nearly identical

Table 3. Training Overhead Comparison.

Method	Peak Memory (MiB)	Time (h)
GRPO	70089	27.1
SAGE-GRPO	70091	30.2
ThinkPrune	69991	29.5
LC-R1	70403	31.8
AdaptThink	75613	33.8

peak memory usage to GRPO. Overall, these results show that SAGE-RL not only improves the main mathematical reasoning benchmarks, but also generalizes to broader reasoning and coding scenarios with competitive training overhead. This supports the central claim that SAGE discovers efficient reasoning patterns that can be internalized by the policy and retained under standard inference.

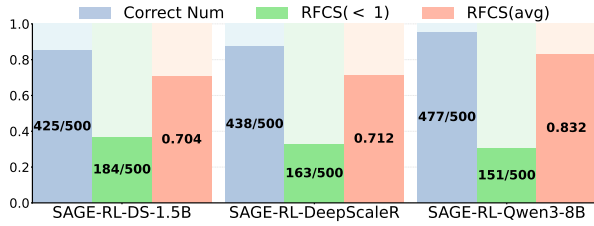


Figure 10. Statistics of RFCS on MATH-500 across different SAGE-RL-tuned models.

7.3. Analysis on Reasoning Behavior

We computed the RFCS metric on MATH-500 for SAGE-GRPO-tuned models, with results shown in Figure 10. Across all models, the proportion of samples with RFCS(<1) decreases substantially compared to Figure 3, indicating a significant reduction in redundant reasoning steps. Simultaneously, the RFCS(avg) increases markedly, suggesting that the reasoning models more frequently terminate thinking immediately after producing the correct answer. As shown in Figure 19 and Figure 20, SAGE-GRPO-tuned models effectively avoid generating a large number of ineffective reasoning steps. These findings strongly confirm that SAGE-RL effectively teaches LRMs precise reasoning patterns.

8. Conclusion

In this work, we uncover and demonstrate that LRMs implicitly know the appropriate time to stop thinking, but this potential is obscured by current sampling paradigms. Built on this observation, We propose SAGE, a sampling paradigm that unleash this capability to uncover precise reasoning chains, yielding significantly CoT length reduction and accuracy improvement. By simply integrating SAGE into the rollout process of RLVR, SAGE-RL achieves lasting gains in inference-time reasoning efficiency.

Impact Statement

This paper uncovers and demonstrates the inherent efficient reasoning potential of LRMs, contributing to the broader field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

Acknowledgements

This research was supported by NSFC (No. 62276015, No. 62506024) and GW2025-09.

References

- Aggarwal, P. and Welleck, S. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*, 2025.
- Aggarwal, P., Madaan, A., Yang, Y., et al. Let’s sample step by step: Adaptive-consistency for efficient reasoning and coding with llms. *arXiv preprint arXiv:2305.11860*, 2023.
- Arora, D. and Zanette, A. Training language models to reason efficiently. *arXiv preprint arXiv:2502.04463*, 2025.
- Arora et al. Training language models to reason efficiently, 2025. URL <https://arxiv.org/abs/2502.04463>.
- Art of Problem Solving. American invitational mathematics examination. https://artofproblemsolving.com/wiki/index.php/American_Invitational_Mathematics_Examination, 2024. Accessed: 2025-03-28.
- Aytes, S. A., Baek, J., and Hwang, S. J. Sketch-of-thought: Efficient llm reasoning with adaptive cognitive-inspired sketching. *arXiv preprint arXiv:2503.05179*, 2025.
- Balachandran, V., Chen, J., Chen, L., Garg, S., Joshi, N., Lara, Y., Langford, J., Nushi, B., Vineet, V., Wu, Y., et al. Inference-time scaling for complex tasks: Where we stand and what lies ahead. *arXiv preprint arXiv:2504.00294*, 2025.
- Cai, W., Liu, Q., and Wang, Y. Learning historical status prompt for accurate and robust visual tracking. *arXiv preprint arXiv:2311.02072*, 7, 2023.
- Cai, W., Liu, Q., and Wang, Y. Hiptrack: Visual tracking with historical prompts. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 19258–19267, 2024.
- Cai, W., Liu, Q., and Wang, Y. Spmtrack: spatio-temporal parameter-efficient fine-tuning with mixture of experts for scalable visual tracking. In *Proceedings of the computer*

- vision and pattern recognition conference, pp. 16871–16881, 2025a.
- Cai, W., Zhu, D., Liu, Q., and Min, Q. Seednorm: Self-rescaled dynamic normalization. *arXiv preprint arXiv:2510.22777*, 2025b.
- Chaoqun, H., Renjie, L., Yuzhuo, B., Shengding, H., Zhen, T., Junhao, S., Jinyi, H., Xu, H., Yujie, H., Yuxiang, Z., Jie, L., Lei, Q., Zhiyuan, L., and Maosong, S. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3828–3850, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.211. URL <https://aclanthology.org/2024.acl-long.211/>.
- Chen, Q., Qin, L., Wang, J., Zhou, J., and Che, W. Unlocking the capabilities of thought: A reasoning boundary framework to quantify and optimize chain-of-thought. *Advances in Neural Information Processing Systems*, 37: 54872–54904, 2024a.
- Chen, R., Zhang, Z., Hong, J., Kundu, S., and Wang, Z. Seal: Steerable reasoning calibration of large language models for free. *arXiv preprint arXiv:2504.07986*, 2025a.
- Chen, W., Wu, Y., Zhang, Z., Zhuang, F., He, Z., Xie, R., and Xia, F. Fairgap: Fairness-aware recommendation via generating counterfactual graph. *ACM Transactions on Information Systems*, 42(4):1–25, 2024b.
- Chen, W., Yuan, M., Zhang, Z., Xie, R., Zhuang, F., Wang, D., and Liu, R. Fairdgccl: Fairness-aware recommendation with dynamic graph contrastive learning. *IEEE Transactions on Knowledge and Data Engineering*, 2025b.
- Chen, W., Guo, X., Li, S., Zhong, Y., Zhang, Z., Zhuang, F., Liu, H., Zhang, L., Ye, G., and He, H. Learning structure-semantic evolution trajectories for graph domain adaptation. *arXiv preprint arXiv:2602.10506*, 2026a.
- Chen, X., Xu, J., Liang, T., He, Z., Pang, J., Yu, D., Song, L., Liu, Q., Zhou, M., Zhang, Z., Wang, R., Tu, Z., Mi, H., and Yu, D. Do NOT think that much for $2+3=?$ on the overthinking of long reasoning models. In *Forty-second International Conference on Machine Learning*, 2025c. URL <https://openreview.net/forum?id=MSbU3L7V00>.
- Chen, Z., Ai, T., Li, Y., Li, G., Wei, Y., Zhou, W., Li, G., Yu, B., Chen, Z., Sun, H., Zhuang, F., Li, J., Wang, D., and Ban, Y. Llmboost: Make large language models stronger with boosting, 2025d. URL <https://arxiv.org/abs/2512.22309>.
- Chen, Z., Li, G., Ai, T., Li, Y., Huang, Z., Zhou, W., Zhuang, F., Liu, X., Li, J., Wang, D., and Ban, Y. Weak-driven learning: How weak agents make strong agents stronger, 2026b. URL <https://arxiv.org/abs/2602.08222>.
- Cheng, Z., Chen, D., Fu, M., and Zhou, T. Optimizing length compression in large reasoning models, 2025. URL <https://arxiv.org/abs/2506.14755>.
- Chuang, Y.-N., Zhou, H., Sarma, P., Gopalan, P., Boccio, J., Bolouki, S., and Hu, X. Learning to route llms with confidence tokens. *arXiv preprint arXiv*, 2410, 2024.
- Cui, Y., He, P., Zeng, J., Liu, H., Tang, X., Dai, Z., Han, Y., Luo, C., Huang, J., Li, Z., et al. Step-wise perplexity-guided refinement for efficient chain-of-thought reasoning in large language models. *arXiv preprint arXiv:2502.13260*, 2025.
- Dai, M., Yang, C., and Si, Q. S-grpo: Early exit via reinforcement learning in reasoning models. *arXiv preprint arXiv:2505.07686*, 2025.
- DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Ding, R., Lv, Y., Meng, X., Song, J., Wang, C., Jiang, C., and Cheng, Y. Prpo: Aligning process reward with outcome reward in policy optimization. *arXiv preprint arXiv:2601.07182*, 2026.
- Fan, C., Zhang, Y., Jia, J., Hero, A., and Liu, S. Cycli-reflex: Improving large reasoning models via cyclical reflection token scheduling, 2025. URL <https://arxiv.org/abs/2506.11077>.
- Fang, Y., Lin, J., Fu, X., Qin, C., Shi, H., Hu, C., Pan, L., Zeng, K., and Cai, X. How to allocate, how to learn? dynamic rollout allocation and advantage modulation for policy optimization. *arXiv preprint arXiv:2602.19208*, 2026a.
- Fang, Y., Lin, J., Fu, X., Qin, C., Shi, H., Liu, C., and Zhao, P. Proximity-based multi-turn optimization: Practical credit assignment for llm agent training. *arXiv preprint arXiv:2602.19225*, 2026b.
- Fu, Z., Fu, Z., Liu, Q., Cai, W., and Wang, Y. Sparsett: Visual tracking with sparse transformers. *arXiv preprint arXiv:2205.03776*, 2022.
- Gao, J., Xu, S., Ye, W., Liu, W., He, C., Fu, W., Mei, Z., Wang, G., and Wu, Y. On designing effective rl reward at training time for llm reasoning, 2024. URL <https://arxiv.org/abs/2410.15115>.

- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Han, T., Wang, Z., Fang, C., Zhao, S., Ma, S., and Chen, Z. Token-budget-aware llm reasoning, 2024.
- Han, T., Wang, Z., Fang, C., Zhao, S., Ma, S., and Chen, Z. Token-budget-aware llm reasoning, 2025. URL <https://arxiv.org/abs/2412.18547>.
- Hassid, M., Synnaeve, G., Adi, Y., and Schwartz, R. Don’t overthink it. preferring shorter thinking chains for improved llm reasoning. *arXiv preprint arXiv:2505.17813*, 2025.
- He, X., Ban, Y., Zou, J., Wei, T., Cook, C., and He, J. Llm-forest: Ensemble learning of llms with graph-augmented prompts for data imputation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 6921–6936, 2025.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- Hou, B., Zhang, Y., Ji, J., Liu, Y., Qian, K., Andreas, J., and Chang, S. Thinkprune: Pruning long chain-of-thought of llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2504.01296>.
- Huang, S., Wang, H., Zhong, W., Su, Z., Feng, J., Cao, B., and Fung, Y. R. Adactrl: Towards adaptive and controllable reasoning via difficulty-aware budgeting, 2025a. URL <https://arxiv.org/abs/2505.18822>.
- Huang, X., Vangani, T. K., Liu, Z., Zou, B., and Aw, A. T. Adacot: Rethinking cross-lingual factual reasoning through adaptive chain-of-thought, 2025b. URL <https://arxiv.org/abs/2501.16154>.
- Huang, Z., Ban, Y., Fu, L., Li, X., Dai, Z., Li, J., and Wang, D. Adaptive sample scheduling for direct preference optimization. *arXiv preprint arXiv:2506.17252*, 2025c.
- Huang, Z., Xia, X., Ren, Y., Zheng, J., Xiao, X., Xie, H., Li, H., Liang, S., Dai, Z., Zhuang, F., Li, J., Ban, Y., and Wang, D. Real-time aligned reward model beyond semantics. 2026. URL <https://api.semanticscholar.org/CorpusID:285240754>.
- Kang, Y., Sun, X., Chen, L., and Zou, W. C3ot: Generating shorter chain-of-thought without compromising effectiveness. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 24312–24320, 2025.
- Kimi Team. Kimi k2: Open agentic intelligence. <https://moonshotai.github.io/Kimi-K2/>, 2025a.
- Kimi Team. Kimi k1.5: Scaling reinforcement learning with llms, 2025b. URL <https://arxiv.org/abs/2501.12599>.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., Brahman, F., Miranda, L. J. V., Liu, A., Dziri, N., Lyu, S., Gu, Y., Malik, S., Graf, V., Hwang, J. D., Yang, J., Bras, R. L., Tafford, O., Wilhelm, C., Soldaini, L., Smith, N. A., Wang, Y., Dasigi, P., and Hajishirzi, H. Tulu 3: Pushing frontiers in open language model post-training, 2025. URL <https://arxiv.org/abs/2411.15124>.
- Lee, A., Che, E., and Peng, T. How well do llms compress their own chain-of-thought? a token complexity approach. *arXiv preprint arXiv:2503.01141*, 2025.
- Lewkowycz, A., Andreassen, A. J., Dohan, D., Dyer, E., Michalewski, H., Ramasesh, V. V., Slone, A., Anil, C., Schlag, I., Gutman-Solo, T., Wu, Y., Neyshabur, B., Gur-Ari, G., and Misra, V. Solving quantitative reasoning problems with language models. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=IFXTZERXdM7>.
- Li, H., Hu, X., and Wang, H. Interpretable unsupervised joint denoising and enhancement for real-world low-light scenarios. *arXiv preprint arXiv:2503.14535*, 2025a.
- Li, H., Wang, Y., Huang, T., Huang, H., Wang, H., and Chu, X. Ld-rps: Zero-shot unified image restoration via latent diffusion recurrent posterior sampling. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 13684–13694, 2025b.
- Li, H., Zhang, W., Hu, X., Jiang, T., Chen, Z., and Wang, H. Prompt-sid: Learning structural representation prompt via latent diffusion for single image denoising. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 4734–4742, 2025c.
- Li, Y., Yuan, P., Feng, S., Pan, B., Wang, X., Sun, B., Wang, H., and Li, K. Escape sky-high cost: Early-stopping self-consistency for multi-step reasoning. *arXiv preprint arXiv:2401.10480*, 2024.

- Li, Z., Xu, T., Zhang, Y., Lin, Z., Yu, Y., Sun, R., and Luo, Z.-Q. Remax: A simple, effective, and efficient reinforcement learning method for aligning large language models. [arXiv preprint arXiv:2310.10505](#), 2023.
- Liao, B., Xu, Y., Dong, H., Li, J., Monz, C., Savarese, S., Sahoo, D., and Xiong, C. Reward-guided speculative decoding for efficient llm reasoning. [arXiv preprint arXiv:2501.19324](#), 2025.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step, 2023. URL <https://arxiv.org/abs/2305.20050>.
- Lin, J., Zhu, C., Kneuert, P. J., Bai, Y., and Xue, Y. Med-causalx: Adaptive causal reasoning with self-reflection for trustworthy medical vision-language models. [arXiv preprint arXiv:2603.23085](#), 2026.
- Liu, T., Guo, Q., Hu, X., Jiayang, C., Zhang, Y., Qiu, X., and Zhang, Z. Can language models learn to skip steps? [arXiv preprint arXiv:2411.01855](#), 2024.
- Liu, Y., Zheng, J., Sun, Z., Peng, Z., Dong, W., Sha, Z., Cui, S., Wang, W., and He, X. Thought manipulation: External thought can be efficient for large reasoning models. [arXiv preprint arXiv:2504.13626](#), 2025.
- Lu, X., Wang, X., Chai, J., Yin, G., Lin, W., Chen, Z., Luo, Y., Zhuang, F., Ban, Y., and Wang, D. Contextual rollout bandits for reinforcement learning with verifiable rewards, 2026. URL <https://arxiv.org/abs/2602.08499>.
- Luo, H., Shen, L., He, H., Wang, Y., Liu, S., Li, W., Tan, N., Cao, X., and Tao, D. O1-pruner: Length-harmonizing fine-tuning for o1-like reasoning pruning. [arXiv preprint arXiv:2501.12570](#), 2025a.
- Luo, M., Tan, S., Wong, J., Shi, X., Tang, W. Y., Roongta, M., Cai, C., Luo, J., Li, L. E., Popa, R. A., and Stoica, I. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. <https://pretty-radio-b75.notion.site/DeepScaler-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902d4468005ed0ca505913a4e2>, 2025b. Notion Blog.
- Luong, M.-T., Hwang, D., Nguyen, H. H., Ghiasi, G., Chervonyi, Y., Seo, I., Kim, J., Bingham, G., Lee, J., Mishra, S., et al. Towards robust mathematical reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 35406–35430, 2025.
- Ma, W., He, J., Snell, C., Griggs, T., Min, S., and Zaharia, M. Reasoning models can be effective without thinking. [arXiv preprint arXiv:2504.09858](#), 2025a.
- Ma, X., Wan, G., Yu, R., Fang, G., and Wang, X. Cot-valve: Length-compressible chain-of-thought tuning. [arXiv preprint arXiv:2502.09601](#), 2025b.
- Manvi, R., Singh, A., and Ermon, S. Adaptive inference-time compute: Llms can predict if they can do better, even mid-generation. [arXiv preprint arXiv:2410.02725](#), 2024.
- Mathematical Association of America. Amc contests. <https://maa.org/student-programs/amc/>, 2023. Accessed: 2025-03-28.
- Meister, C., Vieira, T., and Cotterell, R. Best-first beam search. *TACL*, 2020.
- Meng, Y., Xia, M., and Chen, D. Simpo: Simple preference optimization with a reference-free reward, 2024. URL <https://arxiv.org/abs/2405.14734>.
- Munkhbat, T., Ho, N., Kim, S. H., Yang, Y., Kim, Y., and Yun, S.-Y. Self-training elicits concise reasoning in large language models. [arXiv preprint arXiv:2502.20122](#), 2025.
- Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., and Stoica, I. Routellm: Learning to route llms with preference data, 2024. URL <https://arxiv.org/abs/2406.18665>, 2024.
- OpenAI. Learning to reason with llms. <https://openai.com/research/learning-to-reason-with-llms>, 2025. Accessed: 15 March 2025.
- OpenAI. Introducing openai o3 and o4-mini. <https://openai.com/index/introducing-o3-and-o4-mini/>, April 2025a.
- OpenAI. OpenAI o3: Most advanced reasoning model. <https://openai.com/index/introducing-o3-and-o4-mini/>, April 2025b.
- Qi, P., Liu, Z., Pang, T., Du, C., Lee, W. S., and Lin, M. Optimizing anytime reasoning via budget relative policy optimization, 2025. URL <https://arxiv.org/abs/2505.13438>.
- Qiao, Z., Deng, Y., Zeng, J., Wang, D., Wei, L., Meng, F., Zhou, J., Ren, J., and Zhang, Y. Concise: Confidence-guided compression in step-by-step efficient reasoning, 2025. URL <https://arxiv.org/abs/2505.04881>.
- Qu, Y., Yang, M. Y., Setlur, A., Tunstall, L., Beeching, E. E., Salakhutdinov, R., and Kumar, A. Optimizing test-time compute via meta reinforcement fine-tuning. [arXiv preprint arXiv:2503.07572](#), 2025.

- Qwen Team. Qwq-32b: Embracing the power of reinforcement learning. <https://qwenlm.github.io/blog/qwq-32b/>, 2025.
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. Direct preference optimization: Your language model is secretly a reward model, 2024. URL <https://arxiv.org/abs/2305.18290>.
- Renze, M. and Guven, E. The benefits of a concise chain of thought on problem-solving in large language models. In *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, pp. 476–483. IEEE, 2024.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Shen, Y., Zhang, J., Huang, J., Shi, S., Zhang, W., Yan, J., Wang, N., Wang, K., Liu, Z., and Lian, S. Dast: Difficulty-adaptive slow-thinking for large reasoning models. *arXiv preprint arXiv:2503.04472*, 2025a.
- Shen, Y., Zhang, J., Huang, J., Shi, S., Zhang, W., Yan, J., Wang, N., Wang, K., Liu, Z., and Lian, S. Dast: Difficulty-adaptive slow-thinking for large reasoning models, 2025b. URL <https://arxiv.org/abs/2503.04472>.
- Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., and Wu, C. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256*, 2024.
- Shrivastava, V., Awadallah, A., Balachandran, V., Garg, S., Behl, H., and Papailiopoulos, D. Sample more to think less: Group filtered policy optimization for concise reasoning. *arXiv preprint arXiv:2508.09726*, 2025.
- Song, M., Zheng, M., Li, Z., Yang, W., Luo, X., Pan, Y., and Zhang, F. Fastcurl: Curriculum reinforcement learning with progressive context extension for efficient training rl-like reasoning models, 2025. URL <https://arxiv.org/abs/2503.17287>.
- Team, K., Du, A., Gao, B., Xing, B., Jiang, C., Chen, C., Li, C., Xiao, C., Du, C., Liao, C., et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.
- Wang, T., Li, L., Guo, H., Chen, Y., Li, Y., Wang, Y., Chen, Y., and Chen, G. Anchored policy optimization: Mitigating exploration collapse via support-constrained rectification, 2026. URL <https://arxiv.org/abs/2602.05717>.
- Wen, L., Cai, Y., Xiao, F., He, X., An, Q., Duan, Z., Du, Y., Liu, J., Tang, L., Lv, X., Zou, H., Deng, Y., Jia, S., and Zhang, X. Light-rl: Curriculum sft, dpo and rl for long cot from scratch and beyond, 2025a. URL <https://arxiv.org/abs/2503.10460>.
- Wen, X., Liu, Z., Zheng, S., Ye, S., Wu, Z., Wang, Y., Xu, Z., Liang, X., Li, J., Miao, Z., et al. Reinforcement learning with verifiable rewards implicitly incentivizes correct reasoning in base llms. *arXiv preprint arXiv:2506.14245*, 2025b.
- Wu, S., Xie, J., Zhang, Y., Chen, A., Zhang, K., Su, Y., and Xiao, Y. Arm: Adaptive reasoning model, 2025. URL <https://arxiv.org/abs/2505.20258>.
- Xia, H., Leong, C. T., Wang, W., Li, Y., and Li, W. Token-skip: Controllable chain-of-thought compression in llms. *arXiv preprint arXiv:2502.12067*, 2025.
- Xie, Y., Kawaguchi, K., Zhao, Y., Zhao, J. X., Kan, M.-Y., He, J., and Xie, M. Self-evaluation guided beam search for reasoning. *Advances in Neural Information Processing Systems*, 36:41618–41650, 2023.
- Xu, S., Xie, W., Zhao, L., and He, P. Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*, 2025a.
- Xu, S., Xie, W., Zhao, L., and He, P. Chain of draft: Thinking faster by writing less, 2025b. URL <https://arxiv.org/abs/2502.18600>.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.
- Yang, C., Si, Q., Duan, Y., Zhu, Z., Zhu, C., Li, Q., Lin, Z., Cao, L., and Wang, W. Dynamic early exit in reasoning models, 2025b. URL <https://arxiv.org/abs/2504.15895>.
- Yang, F., Chen, Z., Wang, X., Lu, X., Chai, J., Yin, G., Lin, W., Ma, S., Zhuang, F., Wang, D., Yang, Y., Li, J., and Ban, Y. Your group-relative advantage is biased, 2026. URL <https://arxiv.org/abs/2601.08521>.
- Yang, R., Bai, H., Liu, S., Yu, G., Fan, R., Dang, Y., Zhang, J., Liu, K., Zhu, J., and Chen, P. Specexit: Accelerating large reasoning model via speculative exit. *arXiv preprint arXiv:2509.24248*, 2025c.

- Yeo, E., Tong, Y., Niu, M., Neubig, G., and Yue, X. Demystifying long chain-of-thought reasoning in llms. arXiv preprint arXiv:2502.03373, 2025.
- Yi et al. Shorterbetter: Guiding reasoning models to find optimal inference length for efficient reasoning, 2025. URL <https://arxiv.org/abs/2504.21370>.
- Yu, P., Xu, J., Weston, J., and Kulikov, I. Distilling system 2 into system 1. arXiv preprint arXiv:2407.06023, 2024.
- Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Dai, W., Fan, T., Liu, G., Liu, L., et al. Dapo: An open-source llm reinforcement learning system at scale. arXiv preprint arXiv:2503.14476, 2025.
- Yuan, M., Xiao, Y., Chen, W., Zhao, C., Wang, D., and Zhuang, F. Hyperbolic diffusion recommender model. In Proceedings of the ACM on Web Conference 2025, pp. 1992–2006, 2025.
- Yue, C., Dong, C., Gao, Y., He, H., Chai, J., Yin, G., and Lin, W. Promoting efficient reasoning with verifiable stepwise reward. arXiv preprint arXiv:2508.10293, 2025.
- Zeng, W., Huang, Y., Liu, Q., Liu, W., He, K., Ma, Z., and He, J. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild, 2025. URL <https://arxiv.org/abs/2503.18892>.
- Zhang, J., Lin, N., Hou, L., Feng, L., and Li, J. Adaptthink: Reasoning models can learn when to think, 2025. URL <https://arxiv.org/abs/2505.13417>.
- Zhang et al. Grpo-lead: A difficulty-aware reinforcement learning approach for concise mathematical reasoning in language models, 2025. URL <https://arxiv.org/abs/2504.09696>.
- Zheng, C., Liu, S., Li, M., Chen, X.-H., Yu, B., Gao, C., Dang, K., Liu, Y., Men, R., Yang, A., et al. Group sequence policy optimization. arXiv preprint arXiv:2507.18071, 2025a.
- Zheng, H., Zhou, Y., Bartoldson, B. R., Kailkhura, B., Lai, F., Zhao, J., and Chen, B. Act only when it pays: Efficient reinforcement learning for llm reasoning via selective rollouts. arXiv preprint arXiv:2506.02177, 2025b.
- Zhu, C., Lin, Y., Shao, J., Lin, J., and Wang, Y. Pathology-aware prototype evolution via llm-driven semantic disambiguation for multicenter diabetic retinopathy diagnosis. In Proceedings of the 33rd ACM International Conference on Multimedia, pp. 9196–9205, 2025.
- Zhu, C., Lin, Y., Chen, S., Wang, Y., and Lin, J. Medeyes: Learning dynamic visual focus for medical progressive diagnosis. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 40, pp. 13916–13924, 2026a.
- Zhu, C., Zeng, J., Jiang, J., Lin, J., and Wang, Y. Medsynapse-v: Bridging visual perception and clinical intuition via latent memory evolution, 2026b. URL <https://arxiv.org/abs/2604.26283>.
- Zou, J., Ban, Y., Li, Z., Qi, Y., Qiu, R., Yang, L., and He, J. Transformer copilot: Learning from the mistake log in LLM fine-tuning. In The Thirty-ninth Annual Conference on Neural Information Processing Systems, 2025. URL <https://openreview.net/forum?id=MRvx1T1kNQ>.

A. Related Work

A.1. Stimulating Reasoning Capabilities through Reinforcement Learning

The introduction of OpenAI o1 (OpenAI, 2025) marks a major advance in reasoning performance and the beginning of the LRM era, inspiring efforts to replicate such strong reasoning abilities (Chen et al., 2026b; Zou et al., 2025; Chen et al., 2025d; He et al., 2025). DeepSeek-R1, for example, achieves comparable results using a simple rule-based reward with the group relative policy optimization (GRPO) (Shao et al., 2024) algorithm, and its open-source release has established RLVR (DeepSeek-AI, 2025; Kimi Team, 2025b; Gao et al., 2024; Lambert et al., 2025; Zeng et al., 2025; Wen et al., 2025a; Song et al., 2025) as an effective paradigm for improving LLM reasoning, and Yang et al. (2026) provides a principled theoretical analysis of its advantage estimation. This paradigm simplifies reward design by employing binary 0/1 rewards determined through rule-based correctness evaluation, eliminating the need for separate reward models as required in original GRPO (Shao et al., 2024; DeepSeek-AI, 2025; Schulman et al., 2017; Huang et al., 2026) implementations, thereby substantially reducing memory and computational overhead during RL training.

Subsequent models, including the Kimi K series (Kimi Team, 2025b;a), QwQ (Qwen Team, 2025), and O3 (OpenAI, 2025b), further advance these capabilities. RLVR assigns scores to trajectories based on pre-designed rules, rewarding desirable behaviors and penalizing undesirable ones. This encourages models to generate long CoTs to maximize correctness, fostering advanced reasoning behaviors such as search and backtracking. However, this also engenders a bias toward redundancy over the risk of error, which results in overthinking—wasting computational resources, impairing model performance, and ultimately limiting the practical applicability of LRMs.

In addition to the methods discussed above, a wide range of advanced techniques have been proposed in recent years to address various challenges in representation learning, model optimization, reasoning, and generative modeling. These include progress in interpretable representation learning (Li et al., 2025a), prompt-based structural modeling (Li et al., 2025c), diffusion-driven restoration (Li et al., 2025b), efficient transformer architectures for visual modeling (Fu et al., 2022), prompt-guided sequence modeling (Cai et al., 2023; 2024), parameter-efficient tuning strategies (Cai et al., 2025a), and novel normalization mechanisms for improving model stability (Cai et al., 2025b). Recent studies have also explored prototype-based medical diagnosis and medical vision-language reasoning (Zhu et al., 2025; 2026a; Lin et al., 2026; Zhu et al., 2026b), fairness-aware recommendation and graph domain adaptation (Chen et al., 2024b; 2025b; 2026a; Yuan et al., 2025), as well as efficient reasoning and reward-guided policy optimization for large language models (Yang et al., 2025c; Wang et al., 2026; Ding et al., 2026; Fang et al., 2026b;a). Although these works are designed for different task scenarios, they collectively enrich the toolkit of modern machine learning research and provide useful insights for understanding the generalization and optimization of neural models.

A.2. Explorations in Efficient Reasoning

Overthinking issue is first identified and analyzed by Chen et al. (2025c), who observe that LRMs generate lengthy outputs that neither improve accuracy nor introduce new solution strategies especially for easy prompt. To address this, various works explore efficient reasoning from different angles.

Training-Free Methods typically improve reasoning efficiency through prompting engineering (Han et al., 2024; Xu et al., 2025a; Lee et al., 2025; Renze & Guven, 2024; Chen et al., 2024a; Aytes et al., 2025; Chuang et al., 2024; Ong et al., 2024; Xu et al., 2025b; Huang et al., 2025b; Han et al., 2025), Best-of-N sampling pruning (Xie et al., 2023; Liao et al., 2025) and optimizations (Li et al., 2024; Manvi et al., 2024; Aggarwal et al., 2023), and early-exit (Ma et al., 2025a; Yang et al., 2025b; Fan et al., 2025) mechanisms during reasoning. These approaches cannot fundamentally resolve the issue of redundant reasoning in models, and their effectiveness is often heavily contingent upon the model’s instruction-following capability. In practice, the observed improvements in experiments are typically modest or insignificant.

While SAGE itself is also a training-free algorithm, it essentially serves to unleash the model’s inherent potential for efficient reasoning. This allows the LRMs to select the currently optimal candidate sequence based on its self-aware at each inference iteration step.

Offline Training Methods primarily supervised fine-tuning models with variable-length CoT data (Yu et al., 2024; Kang et al., 2025; Xia et al., 2025; Ma et al., 2025b; Munkhbat et al., 2025; Liu et al., 2024; Han et al., 2024). Recently, ConCISE (Qiao et al., 2025) constructs concise CoT data by inserting prompt tokens and employing early-exit during inference, then enhances the model’s reasoning conciseness through SFT/SimPO (Rafailov et al., 2024; Meng et al., 2024). The primary

challenge of this line of work lies in the difficulty of obtaining high-quality short chains of thought, and the offline training paradigm tends to limit the model’s exploration ability on difficult problems.

For similar reasons, we do not choose offline distillation to learn trajectories sampled by SAGE in this work. Since distillation depends on a strong teacher model, we are concerned that self-distillation will limit the upper boundary of the model’s reasoning capability.

Online Training Methods mainly adopt reinforcement learning for better generalization. (Kimi Team, 2025b; Shen et al., 2025b; Cheng et al., 2025; Team et al., 2025; Luo et al., 2025a; Aggarwal & Welleck, 2025; Arora & Zanette, 2025; Yeo et al., 2025; Shen et al., 2025a; Qu et al., 2025; Cui et al., 2025) introduce length penalties in the reward function to suppress overly long reasoning traces. Yi et al. (2025), Hou et al. (2025), and Qi et al. (2025) optimize performance under a fixed token budget to balance efficiency and effectiveness. GFPO (Shrivastava et al., 2025) attains sampling outputs aligned with the optimization objective via oversampling. S-GRPO (Dai et al., 2025) and VSRM (Yue et al., 2025) truncate reasoning steps and perform repeated rollouts to evaluate the rewards of reasoning subchains, which are then leveraged for RL training. Zhang et al. (2025), Huang et al. (2025a), and Wu et al. (2025) assign predefined thinking patterns based on task difficulty, which essentially reflects a length budget. All the aforementioned methods are heavily rely on sophisticated reward design, which can easily lead to training instability or even reward hacking during the RL training process. Moreover, explicit or implicit integration of length compression into the optimization objective may impair the model’s reasoning capabilities.

In this work, instead of modifying the optimization objective, we optimize the sampling process to enable the policy model to directly learn the efficient reasoning chains uncovered by SAGE via the advantage estimation of RLVR. This design yields the following two key advantages: **(1) Low Computational Cost:** We eliminate the need for extra oversampling as in GFPO (Shrivastava et al., 2025), where a single parallel sampling step suffices to generate high-quality reasoning chains. Additionally, we do not require repeated rollouts for reward value estimation, a step essential to methods such as S-GRPO (Dai et al., 2025) and VSRM (Yue et al., 2025). **(2) Stable Training Dynamics:** By preserving all components of RLVR except for the rollout procedure, SAGE-RL exhibits no significant difference in training stability compared with vanilla RLVR.

B. Significant Differences from Beam Search

In this section, we highlight the significant distinctions between TSearch w/ Φ and Beam Search from two perspectives: experimental results and underlying principles.

Table 4. Performance Comparison of different sampling strategies on different models (Max Tokens=10,086). Due to the inherent characteristic of Beam Search that it returns multiple responses by default, when calculating the ACC of Beam Search and TSearch, we consider a result correct if it contains at least one correct answer.

Model	Sampling Strategy	ACC	LEN
DS-1.5B	Greedy	0.81	4216
	Random	0.81	<u>4142</u>
	Beam Search (4, 4)	<u>0.82</u>	4472
	TSearch w/ Φ (4, 4)	0.84	2972
Qwen3-8B	Greedy	0.82	<u>4505</u>
	Random	0.82	4526
	Beam Search (4, 4)	<u>0.84</u>	4655
	TSearch w/ Φ (4, 4)	0.89	2946

We compared the performance of vanilla beam search with TSearch w/ Φ on a randomly selected subset of MATH-500 (size=100). For the fairness of comparison, we uniformly set the exploration width to $m = 4$. Since Beam Search directly returns the final set of candidate sequences, i.e., the number of returned sequences $r = m$, we therefore uniformly set $r = 4$. As shown in Table 4, Even though Beam Search generates four responses for each question, its final ACC is only comparable to those of random sampling and greedy sampling. Conversely, our algorithm achieves markedly higher accuracy while significantly reducing average response length.

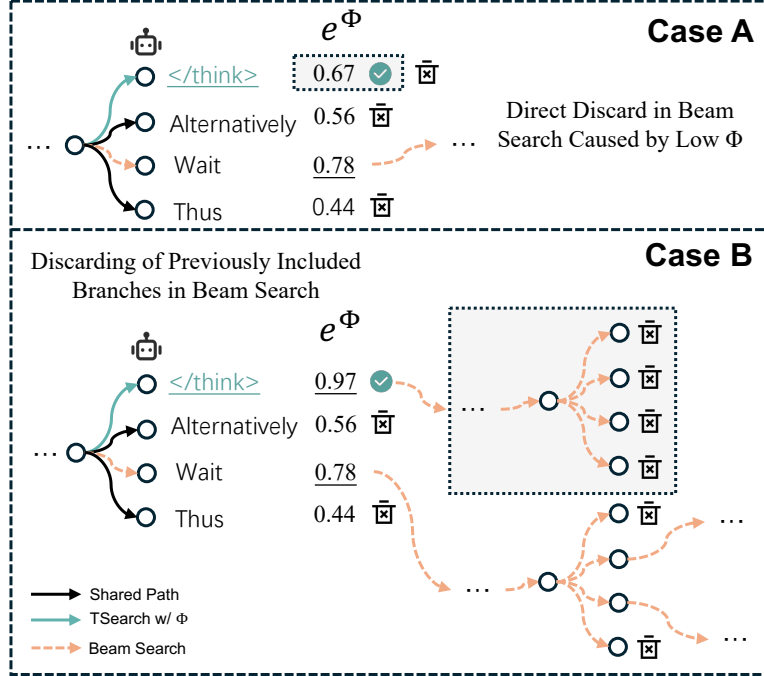


Figure 11. Two distinctions between TSearch w/ Φ and vanilla beam search.

We analyze and illustrate the root causes of these differences in Figure 11. **In Case A**, although `</think>` appears within the log-probability window, the corresponding candidate sequence is discarded because its overall confidence score Φ does not rank first. **In Case B**, a candidate sequence containing `</think>` is initially retained but is subsequently pruned during further expansion. In contrast, our algorithm directly accepts the sequence upon detecting `</think>`. These results indicate that our algorithm prevents the premature discarding of precise reasoning branches in later steps and significantly enhancing reasoning efficiency.

C. The Motivation of Transition from Token-wise to Step-wise

Intuitively, steps serve as the logical reasoning units in LRMs, thus token-wise and step-wise search behave highly consistently. We further verify this via the following analyses.

Termination Behavior: We clarify that a reasoning step is defined as a complete reasoning segment generated by the model, ending at a natural step boundary or the end-of-thinking token. Since the end-of-thinking token appears at the end of a reasoning step rather than at an arbitrary token position, token-wise and step-wise exploration are aligned for the termination decision.

Intermediate behavior: We also examined whether token-level exploration inside a reasoning step is necessary. For each problem, we sampled complete-step prefixes and random-position prefixes, then ran TSearch from these prefixes and encoded candidates with all-MiniLM-L6-v2. Candidate branches starting from random positions inside a step were substantially more similar than those starting from complete steps, indicating that token-level search inside a step contributes little diversity.

Overall performance: Finally, we compared token-wise TSearch and step-wise SAGE under aligned settings. As Table 6 shows, SAGE closely matches TSearch in accuracy while avoiding expensive token-level candidate maintenance.

In summary, step-wise search behaves consistently with token-wise search, while lowering overhead by reducing candidate sequence maintenance. Thus, switching from TSearch to SAGE is natural and reasonable.

Table 5. Similarity among branches generated from different prefix types. Higher similarity for random-position prefixes suggests that token-level search inside reasoning steps is less useful than step-level branching.

Prefix Type	Similarity
Complete steps	0.6883
Random positions	0.8897

Table 6. Token-wise TSearch and step-wise SAGE under aligned settings.

Method	TR	ACC	T-LEN	LEN
TSearch (4,1) w/ Φ	1.00	0.92	2213	2609
SAGE (4,1)	–	0.92	2234	2655

D. Experimental Details

D.1. Objectives and Training Hyperparameters

The objectives of GRPO and SAGE-GRPO are as follows:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min(w_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(w_{i,t}(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_{i,t}) \right], \quad (9)$$

$$\begin{aligned} \mathcal{J}_{\text{SAGE-GRPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \left(\underbrace{\sum_{i=1}^r \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min(w_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(w_{i,t}(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_{i,t})}_{\text{SAGE (m, r)}} + \right. \right. \\ \left. \left. \underbrace{\sum_{i=r+1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min(w_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(w_{i,t}(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_{i,t})}_{\text{Random Sampling}} \right) \right] \quad (10) \end{aligned}$$

where G is the number of generated responses to each query x (i.e., the group size), and the importance ratio $w_{i,t}(\theta)$ and advantage $\hat{A}_{i,t}$ of token $y_{i,t}$ are:

$$w_{i,t}(\theta) = \frac{\pi_{\theta}(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})}, \hat{A}_{i,t} = \hat{A}_i = \frac{r(x, y_i) - \text{mean}(\{r(x, y_i)\}_{i=1}^G)}{\text{std}(\{r(x, y_i)\}_{i=1}^G)}, \quad (11)$$

The objectives of GSPO and SAGE-GSPO are as follows:

$$\mathcal{J}_{\text{GSPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \min(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i) \right], \quad (12)$$

$$\begin{aligned} \mathcal{J}_{\text{SAGE-GSPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \left(\underbrace{\sum_{i=1}^r \min(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i)}_{\text{SAGE (m, r)}} + \right. \right. \\ \left. \left. \underbrace{\sum_{i=r+1}^G \min(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i)}_{\text{Random Sampling}} \right) \right] \quad (13) \end{aligned}$$

where we adopt the group-based advantage estimation:

$$\hat{A}_i = \frac{r(x, y_i) - \text{mean}(\{r(x, y_i)\}_{i=1}^G)}{\text{std}(\{r(x, y_i)\}_{i=1}^G)}, \quad (14)$$

and define the importance ratio $s_i(\theta)$ based on sequence likelihood:

$$s_i(\theta) = \left(\frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)} \right)^{\frac{1}{|y_i|}} = \exp \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})} \right). \quad (15)$$

D.2. Experimental Setup

To thoroughly evaluate the effectiveness of SAGE-RL, we conduct experiments using several widely adopted LRMs as base models, including DeepSeek-R1-Distill-Qwen-1.5B (DS-1.5B), DeepSeek-R1-Distill-Qwen-7B (DS-7B) (DeepSeek-AI, 2025), DeepScaleR (Luo et al., 2025b), and Qwen3-8B (Yang et al., 2025a).

Training Data Considering the importance of training data quality (Huang et al., 2025c), we use the English subset of DAPO (Yu et al., 2025) as well as MATH (Hendrycks et al., 2021) problems with difficulty from level 3 to level 5 (Zheng et al., 2025b). This collection consists of approximately 20,000 carefully curated problems covering a wide range of difficulty levels.

Training Configuration We use the ver1 (Sheng et al., 2024) framework for SAGE-RL training using the rule based reward function. To ensure a completely fair comparison that highlights the role of SAGE in the rollout phase, we adopt identical hyperparameter settings for the same base model across SAGE-RL and all its baselines and variants. We tune the base models with a global batch size of 32 across 8 GPUs for 600 steps with the Adam optimizer with learning rate of 1e-6, cosine warmup for the first 50 steps, and sampling temperature $T = 1.0$. We apply KL regularization with $\beta = 0.001$ and an entropy coefficient of $\gamma = 0.001$. Our models are trained with 9,216 maximum context length, with 1,024 tokens reserved for the prompt.

Sampling Strategy We tune all models with a group size of $G = 8$. Within each group, SAGE-RL employs SAGE (2,2) to search for two completions with precise reasoning chains, while the remaining six completions are obtained through the default random sampling in ver1.

Evaluation We follow previous work (Yue et al., 2025; Liu et al., 2025; Dai et al., 2025) and select a comprehensive set of benchmarks, AIME24, AIME25(Art of Problem Solving, 2024), OlympiadBench(Chaoqun et al., 2024), MATH-500, Minerva(Lewkowycz et al., 2022), and AMC23(Mathematical Association of America, 2023), providing broader coverage than previous studies. During evaluation, we set the maximum generation length at 32768 tokens, consistent with Hou et al. (2025)’s work and DeepSeek-R1. The temperature and top-p are set to 1.0 and 0.95, respectively. For all benchmarks, we report the average pass@1, response length(LEN) and token efficiency(TE) over N runs. Specifically, for OlympiadBench, Minerva and MATH-500 where the benchmark sizes are relatively large, we set N to 8; for the other benchmarks, we set N to 32 to reduce randomness.

E. Additional Experimental Results

E.1. Comparison with Extended Datasets and Additional Analysis

In this section, we present the complete evaluation results on six mathematical datasets. We divide the benchmarks into two groups of equal size. The three datasets in the upper part of Table 7 are more challenging than those in the lower part.

Performance on DS-1.5B and DS-7B For experiments with DS-1.5B as the base model, SAGE-RL consistently achieve the best or second-best performance across all benchmarks, while effectively improving the original model’s capabilities across all six mathematical reasoning benchmarks. Notably, SAGE-GSPO yields significant pass@1 gains of 6.2 % on AIME 2025 and 8.2 % on AMC23. AdaptThink stands out as a powerful baseline, attaining the highest token efficiency on MATH-500 and Minerva, while exhibiting the most pronounced reasoning simplification on OlympiadBench, MATH-500, and Minerva, but this high level of conciseness restrict the model’s ability to explore different solution strategies. As a result, AdaptThink struggle in terms of overall performance and consistently lag behind our method. As for the other baselines, both their performance and efficiency are generally less competitive compared to SAGE-RL.

Does Your Reasoning Model Implicitly Know When to Stop Thinking?

Table 7. Pass@1, response length(LEN) and TE results on six benchmarks and four base models before and after LC-R1, ThinkPrune-2k, AdaptThink, Efficient Reasoning, GRPO-LEAD, GRPO, GSPO, SAGE-GRPO and SAGE-GSPO. TE is calculated as Pass@1/LEN. Bold and underlined numbers denote the best and second-best results. The percentage in parentheses after TE indicates the improvement compared with the base model.

Method	AIME 2024			AIME 2025			OlympiadBench		
	Pass@1↑(%)	LEN↓	TE↑(×10 ⁻³)	Pass@1↑(%)	LEN↓	TE↑(×10 ⁻³)	Pass@1↑(%)	LEN↓	TE↑(×10 ⁻³)
DS-1.5B	25.1	12300	2.04	20.9	11669	1.79	33.4	8954	3.73
+ LC-R1	23.3 (11.8)	7098 (15202)	3.28 (160.8%)	20.9 (10.0)	6942 (14727)	3.01 (168.2%)	32.0 (11.4)	4632 (14322)	6.91 (185.3%)
+ ThinkPrune-2k	23.7 (11.4)	<u>7085</u> (15215)	3.35 (164.2%)	19.7 (11.2)	6918 (14751)	2.85 (159.2%)	32.9 (10.5)	4752 (14202)	6.92 (185.5%)
+ AdaptThink	25.7 (10.6)	8055 (14245)	3.19 (156.4%)	21.8 (10.9)	8155 (13514)	2.67 (149.2%)	32.6 (10.8)	4563 (14391)	7.14 (191.4%)
+ Efficient Reasoning	26.2 (11.1)	9189 (13111)	2.85 (139.7%)	22.9 (12.0)	8590 (13079)	2.67 (149.2%)	33.8 (10.4)	5755 (13199)	5.87 (157.4%)
+ GRPO	28.3 (13.2)	8767 (13533)	3.23 (158.3%)	24.1 (13.2)	8263 (13406)	2.92 (163.1%)	34.2 (10.8)	6323 (12631)	5.41 (145.0%)
+ SAGE-GRPO	28.8 (13.7)	7243 (15057)	<u>3.98</u> (195.1%)	<u>26.5</u> (15.6)	7479 (14190)	<u>3.54</u> (197.8%)	<u>36.9</u> (13.5)	<u>5050</u> (13904)	7.31 (196.0%)
+ GSPO	28.3 (13.2)	8604 (13696)	3.29 (161.3%)	25.1 (14.2)	8227 (13442)	3.05 (170.4%)	34.6 (11.2)	6410 (12544)	5.40 (144.8%)
+ SAGE-GSPO	<u>28.5</u> (13.4)	6889 (15411)	4.14 (1102.9%)	27.1 (16.2)	<u>7167</u> (14502)	3.78 (1111.1%)	37.3 (13.9)	5172 (13782)	<u>7.21</u> (193.3%)
DeepScaleR	31.4	9370	3.35	25.4	9310	2.73	35.9	5972	6.01
+ ThinkPrune-2k	33.5 (12.1)	<u>8108</u> (11262)	4.13 (123.3%)	26.0 (10.6)	7486 (11824)	<u>3.47</u> (127.1%)	35.1 (10.8)	4723 (11249)	<u>7.43</u> (123.6%)
+ GRPO	<u>35.6</u> (14.2)	8592 (1778)	<u>4.14</u> (123.6%)	27.4 (12.0)	8185 (11125)	3.35 (122.7%)	<u>36.2</u> (10.3)	5443 (1529)	6.65 (110.6%)
+ SAGE-GRPO	36.1 (14.7)	8094 (11276)	4.46 (133.1%)	<u>27.2</u> (11.8)	<u>7704</u> (11606)	3.53 (129.3%)	36.5 (10.6)	4890 (11082)	7.46 (124.1%)
DS-7B	51.9	11305	4.59	37.1	12540	2.96	39.8	7839	5.08
+ LC-R1	51.7 (10.2)	6820 (14485)	7.58 (165.1%)	35.7 (11.4)	<u>7458</u> (15082)	<u>4.79</u> (161.8%)	<u>41.4</u> (11.6)	4193 (13646)	9.87 (194.3%)
+ AdaptThink	52.1 (10.2)	6679 (14626)	<u>7.80</u> (169.9%)	35.0 (12.1)	7807 (14733)	4.48 (172.3%)	38.9 (10.9)	4915 (12924)	7.91 (155.7%)
+ Efficient Reasoning	51.9 (10.0)	<u>6667</u> (14638)	7.78 (169.5%)	36.2 (10.9)	7501 (15039)	4.82 (162.8%)	40.1 (10.3)	4599 (13240)	8.72 (171.7%)
+ GRPO-LEAD	53.1 (11.2)	7023 (14282)	7.56 (164.7%)	36.1 (11.0)	7842 (14698)	4.60 (155.4%)	40.6 (10.8)	4972 (12867)	8.17 (160.8%)
+ GRPO	52.5 (10.6)	8424 (12881)	6.23 (135.7%)	38.4 (11.3)	10123 (12417)	3.79 (128.0%)	41.2 (11.4)	5498 (12341)	7.50 (147.6%)
+ SAGE-GRPO	55.3 (13.4)	6422 (14883)	8.61 (187.6%)	<u>38.0</u> (10.9)	6583 (15957)	5.77 (194.9%)	41.8 (12.0)	<u>4435</u> (13404)	<u>9.42</u> (185.4%)
Qwen3-8B	73.2	15920	4.60	67.3	18342	3.67	<u>46.6</u>	11707	4.00
+ GRPO	72.8 (10.4)	10573 (15347)	6.89 (149.8%)	<u>66.6</u> (10.7)	13981 (14361)	4.76 (129.7%)	45.1 (11.5)	7512 (14195)	6.00 (150.0%)
+ SAGE-GRPO	<u>73.5</u> (10.3)	8975 (16945)	<u>8.19</u> (178.0%)	<u>66.6</u> (10.7)	<u>10052</u> (18290)	<u>6.58</u> (179.3%)	45.4 (11.2)	<u>5972</u> (15735)	<u>7.60</u> (190.0%)
+ GSPO	73.0 (10.2)	10544 (15376)	6.92 (150.4%)	66.2 (11.1)	<u>14082</u> (14260)	<u>4.70</u> (130.2%)	<u>46.6</u> (10.0)	7964 (13743)	5.85 (146.2%)
+ SAGE-GSPO	73.7 (10.5)	8547 (17373)	8.62 (187.4%)	66.0 (11.3)	9183 (19159)	7.19 (195.9%)	46.7 (10.1)	5436 (16271)	8.59 (1114.7%)
Method	MATH-500			Minerva			AMC23		
	Pass@1↑(%)	LEN↓	TE↑(×10 ⁻³)	Pass@1↑(%)	LEN↓	TE↑(×10 ⁻³)	Pass@1↑(%)	LEN↓	TE↑(×10 ⁻³)
DS-1.5B	83.2	4882	17.0	30.1	6210	4.85	60.1	8250	7.28
+ LC-R1	80.4 (12.8)	2973 (11909)	27.0 (158.8%)	31.8 (11.7)	<u>3512</u> (12698)	9.06 (186.8%)	61.8 (11.7)	4889 (13361)	12.6 (173.6%)
+ ThinkPrune-2k	81.7 (11.5)	2826 (12056)	28.9 (170.0%)	32.9 (12.8)	3667 (12543)	8.97 (185.0%)	60.8 (10.7)	5224 (13026)	11.6 (159.9%)
+ AdaptThink	80.4 (12.8)	2563 (12319)	31.4 (184.1%)	32.3 (12.2)	2912 (13298)	11.1 (1128.7%)	62.3 (12.2)	4969 (13281)	12.5 (171.7%)
+ Efficient Reasoning	82.0 (11.2)	<u>2821</u> (12061)	29.1 (170.6%)	31.4 (11.3)	3530 (12680)	8.90 (183.5%)	64.7 (14.6)	5202 (13048)	12.4 (170.9%)
+ GRPO	83.6 (10.4)	3907 (1975)	21.4 (125.6%)	32.0 (11.9)	4806 (11404)	6.66 (137.3%)	65.4 (15.3)	5771 (12479)	11.3 (155.6%)
+ SAGE-GRPO	<u>84.8</u> (11.6)	2915 (11967)	29.1 (170.7%)	33.8 (13.7)	3735 (12475)	9.05 (186.6%)	<u>66.3</u> (16.2)	5091 (13159)	13.0 (178.9%)
+ GSPO	83.4 (10.2)	3898 (1984)	25.3 (121.4%)	32.0 (11.9)	4454 (11756)	7.18 (148.0%)	66.1 (16.0)	6095 (12191)	10.9 (163.1%)
+ SAGE-GSPO	85.2 (12.0)	2921 (11961)	<u>29.2</u> (171.6%)	<u>33.6</u> (13.5)	3647 (12563)	<u>9.21</u> (189.9%)	68.3 (18.2)	5278 (12972)	<u>12.9</u> (177.7%)
DeepScaleR	86.0	3805	22.6	38.6	5184	7.45	64.2	6683	9.61
+ ThinkPrune-2k	82.5 (13.5)	2946 (1859)	<u>28.0</u> (123.9%)	37.9 (10.7)	3188 (11996)	11.9 (159.6%)	65.8 (11.6)	5046 (11637)	<u>13.0</u> (135.6%)
+ GRPO	87.6 (11.6)	3482 (1323)	25.2 (111.3%)	40.4 (11.8)	4386 (1798)	9.21 (123.6%)	<u>69.3</u> (15.1)	5872 (1811)	11.8 (122.8%)
+ SAGE-GRPO	88.8 (12.8)	<u>3117</u> (1688)	28.4 (125.7%)	41.4 (12.8)	<u>3817</u> (11367)	<u>10.9</u> (145.6%)	70.9 (16.7)	<u>5438</u> (11245)	13.0 (135.7%)
DS-7B	91.6	3871	23.7	43.0	5490	7.83	81.9	7170	11.4
+ LC-R1	87.3 (14.3)	2076 (11795)	<u>42.1</u> (177.7%)	44.4 (11.4)	2834 (12656)	15.7 (1100.0%)	79.1 (12.8)	3686 (13484)	<u>21.5</u> (187.9%)
+ AdaptThink	88.9 (12.7)	2199 (11672)	40.4 (170.9%)	45.2 (12.2)	<u>2869</u> (12621)	<u>15.8</u> (1101.2%)	80.7 (11.2)	5130 (12040)	15.7 (137.7%)
+ Efficient Reasoning	89.8 (11.8)	2408 (11463)	37.3 (157.6%)	45.7 (12.7)	2903 (12587)	15.7 (1101.0%)	80.7 (11.2)	4933 (12237)	16.4 (143.3%)
+ GRPO-LEAD	89.5 (12.1)	2752 (11119)	32.5 (137.1%)	46.3 (13.3)	2990 (12500)	16.0 (1104.3%)	82.7 (10.8)	4384 (12786)	18.9 (165.8%)
+ GRPO	<u>92.0</u> (10.4)	3219 (1652)	28.5 (120.2%)	<u>46.0</u> (13.0)	3510 (11980)	13.1 (167.4%)	<u>83.0</u> (11.1)	4880 (12290)	17.0 (149.0%)
+ SAGE-GRPO	93.0 (11.4)	<u>2141</u> (11730)	43.4 (183.1%)	45.2 (12.2)	2692 (12798)	16.8 (1114.4%)	84.9 (13.0)	<u>3953</u> (13217)	21.5 (188.1%)
Qwen3-8B	94.4	5640	16.7	51.8	7358	7.04	90.5	10852	8.34
+ GRPO	93.6 (10.8)	4470 (11170)	20.9 (125.1%)	52.6 (10.8)	4964 (12394)	10.6 (150.6%)	88.6 (11.9)	7079 (13773)	12.5 (150.1%)
+ SAGE-GRPO	95.0 (10.6)	<u>3015</u> (12625)	<u>31.5</u> (188.2%)	<u>53.5</u> (11.7)	<u>3390</u> (13968)	<u>15.8</u> (1124.2%)	<u>90.7</u> (10.2)	<u>5563</u> (15289)	<u>16.3</u> (195.4%)
+ GSPO	94.6 (10.2)	4342 (11298)	22.2 (132.9%)	49.6 (12.2)	3962 (13396)	12.5 (177.8%)	87.7 (12.8)	6464 (14388)	13.6 (163.1%)
+ SAGE-GSPO	94.4 (10.0)	2753 (12887)	34.3 (1105.3%)	53.7 (11.9)	3363 (13995)	16.0 (1126.8%)	90.9 (10.4)	5041 (15811)	18.0 (1106.7%)

A similar trend is observed when using DS-7B as the base model. Both GRPO-LEAD and Efficient-Reasoning adopt a strategy of sacrificing less compression in exchange for improved performance; however, the reasoning capability gains they achieve remain substantially smaller than those of SAGE-RL. For instance, on AIME 2024, SAGE-GRPO not only outperforms GRPO-LEAD by 2.2 % in pass@1, but also produces noticeably shorter responses.

These two sets of experiments together indicate that, on distilled models, SAGE-RL not only effectively alleviates the overthinking problem but also substantially enhances the model’s reasoning capability on complex mathematical problems. Since our method simultaneously improves the base model’s reasoning capability and the precision of its thinking process, it achieves more consistent and substantially larger gains in token efficiency compared to other approaches.

Performance on DeepScaleR DeepScaleR has undergone systematic and comprehensive reinforcement learning (Luo et al., 2025b). Consequently, additional fine-tuning on this model typically yields only marginal performance gains, which accounts for the scarcity of related works that adopt DeepScaleR as a base model. Nevertheless, SAGE-RL still achieves relatively significant token efficiency improvements on DeepScaleR. Particularly on OlympiaBench, Math-500 and Minerva, SAGE-RL delivers roughly twice the token efficiency gains of GRPO, demonstrating that SAGE-RL yields relatively substantial benefits even for models with extensive post-training.

Performance on Qwen3-8B As one of the strongest reasoning models under the same parameter scale, Qwen3-8B achieves excellent performance across various mathematical reasoning tasks. Even on the highly challenging AIME 2025, it attains an impressive pass@1 of 67.3%. However, as illustrated in Figure 3, the overthinking problem remains largely unresolved in this model. For instance, on MATH-500, despite comparable pass@1 performance, the average response length is more than 2.5 times that of SAGE-GRPO-tuned DS-7B.

Notably, vanilla RLVR is capable of moderately reducing the response length of the base model. This effect stems from the training procedure, where sequences must be padded to a fixed batch length, causing the token budget in evaluation to be significantly smaller than that used in inference. As a result, the model tends to receive positive rewards more readily for short answers, encouraging shorter generations. Nevertheless, this mechanism limits the model’s ability to improve or may even cause declines on reasoning tasks of varying difficulty, particularly on datasets such as AIME 2024 and AMC 23.

In contrast, SAGE-RL still achieves moderate improvements in the reasoning capability of Qwen3-8B under limited training token budgets, while effectively reducing the redundancy in the thinking process. For example, SAGE-GSPO attains a 1.9% increase in pass@1 on Minerva, while compressing the average response length to only 45.7% of the original. These results strongly demonstrate that SAGE-RL remains highly effective even on state-of-the-art reasoning models.

Comparison of SAGE-GRPO and SAGE-GSPO As shown in Figure 9, across both GRPO and GSPO, the key variations in pass@1, response length, and KL loss are driven by the incorporation of SAGE-RL rather than fundamental RLVR algorithms. This underscores the robust positive impact of our approach across different RLVR implementations. In terms of entropy, GSPO-tuned models show elevated values, largely due to sequence-level importance sampling disregarding fine-grained token-level variations, thereby resulting in higher inference uncertainty.

From the experimental results shown in Table 7, SAGE-GSPO exhibits particularly strong performance in reducing response length and slightly outperforms SAGE-GRPO in overall metrics. We hypothesize that this advantage stems from the greater stability of GSPO’s sequence-level importance sampling compared to GRPO’s token-level importance sampling, which is especially beneficial in more unstable scenarios such as MoE models (Zheng et al., 2025a).

A similar issue arises in SAGE-RL due to the hybrid sampling used in the rollout phase. In SAGE-GRPO, some of the rollouts are generated by selecting sequences at every reasoning step based on the full-sequence confidence score Φ , rather than greedily choosing the highest log-probability token as in Equation 7. Consequently, as indicated in Equation 11, the probability $\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})$ under the old policy may be lower than that of random sampling, increasing the likelihood of clipping during importance sampling. In contrast, GSPO treats the entire sequence as the basic unit for importance sampling, thereby avoiding this issue entirely.

Overall, Table 7 demonstrates that SAGE-RL achieves substantially superior performance compared to all baseline methods across six challenging mathematical reasoning tasks. Meanwhile, Figure 9 reveals higher pass@1 scores and increased KL divergence, accompanied by reduced response entropy and shorter response lengths. These results indicate that SAGE successfully unleashes the model’s implicit capacity for timely thinking termination. Consequently, the model learns efficient reasoning with increased confidence, confirming the viability of leveraging RLVR to instill effective reasoning patterns. This is consistent with the results of Wen et al. (2025b), which demonstrate that RLVR effectively promotes correct reasoning chains in base LLMs.

E.2. Hyperparameters Sensitivity Analysis

This section examines the influence of the two primary hyperparameters influencing SAGE-RL: the SAGE exploration width m and the total number of rollouts r produced by SAGE per group. We evaluate SAGE-RL under various combinations of these parameters and denote each setting as SAGE(m, r)-RL. Figure 12 illustrates the training dynamics of DS-1.5B with SAGE-GRPO under different hyperparameter combinations and GRPO. The corresponding evaluation results on four mathematical datasets are reported in Table 8.

Table 8. A comparison of experimental results for DS-1.5B under different SAGE-GRPO parameter settings. Here, SAGE (m, r) denotes an exploration width of m , with the final retention of r different trajectories.

Method	MATH-500			AIME 2024			AIME 2025			OlympiadBench		
	Pass@1↑(%)	LEN↓	TE↑($\times 10^{-3}$)	Pass@1↑(%)	LEN↓	TE↑($\times 10^{-3}$)	Pass@1↑(%)	LEN↓	TE↑($\times 10^{-3}$)	Pass@1↑(%)	LEN↓	TE↑($\times 10^{-3}$)
DS-1.5B	83.2	4882	17.0	25.1	12300	2.04	20.9	11669	1.79	33.4	8954	3.73
+ GRPO	83.6 (10.4)	3907 (1975)	21.4 (125.6%)	28.3 (13.2)	8767 (13533)	3.23 (158.3%)	24.1 (13.2)	8263 (13406)	2.92 (163.1%)	34.2 (10.8)	6323 (12631)	5.41 (145.0%)
+ SAGE (1,1)-GRPO	84.0 (10.8)	3416 (11466)	24.6 (144.7%)	28.3 (13.2)	7979 (14321)	3.55 (174.0%)	24.8 (13.9)	7730 (13939)	3.21 (179.3%)	34.5 (11.1)	5857 (13097)	5.89 (157.9%)
+ SAGE (2,1)-GRPO	84.2 (11.0)	2952 (11930)	28.5 (167.8%)	28.5 (13.4)	7308 (14992)	3.90 (191.2%)	25.7 (14.8)	7603 (14066)	3.38 (188.8%)	35.2 (11.8)	5267 (13687)	6.68 (179.1%)
+ SAGE (2,2)-GRPO	84.8 (11.6)	2915 (11967)	29.1 (170.7%)	28.8 (13.7)	7243 (15057)	3.98 (195.1%)	26.5 (15.6)	7479 (14190)	3.54 (197.8%)	36.9 (13.5)	5050 (13904)	7.31 (196.0%)

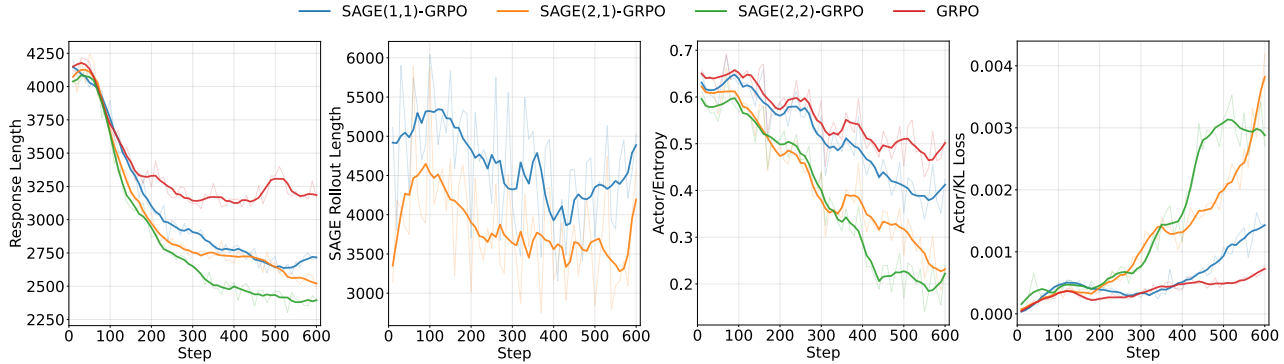


Figure 12. Training dynamics comparison for SAGE-GRPO with distinct hyperparameter combination: average response length when tested on MATH500, average SAGE-produced trajectory length during training, entropy, and KL divergence.

The Impact of SAGE Rollout Quantity As shown in Table 8, the transition of r from 1 to 2 has limited effect on the results. From a policy optimization perspective, larger r allows the policy model to learn from more efficient reasoning samples; however, the advantage estimate per sample becomes less sharp compared to $r = 1$, leading to similar overall updates.

Figure 12 shows that SAGE (2,1)-GRPO and SAGE (2,2)-GRPO display very similar trends in entropy and KL divergence, markedly different from those of SAGE (1,1)-GRPO and vanilla GRPO. This indicates that **enlarging r has little impact on policy updates, as rollouts with similar reasoning trajectories offer minimal additional information.**

The Impact of Exploration Width On the other hand, enlarging m from 1 to 2 yields substantial performance gains. According to the results shown in Table 8, while SAGE (1,1)-GRPO yields moderate improvements over the vanilla GRPO baseline, its performance is markedly inferior to that of SAGE (2,1)-GRPO.

This indicates that exploration width significantly influences the activation of the model’s efficient reasoning capability, consistent with the findings in Figure 11. As illustrated in Figure 12, SAGE (1,1)-GRPO exhibits significantly milder entropy reduction and KL divergence increase relative to SAGE (2,1)-GRPO, and its training dynamics remain much closer to the vanilla GRPO. More directly, both the average length of SAGE-produced rollouts in SAGE(2,1)-GRPO and the average

response length of the model at test time are significantly shorter than those observed in SAGE(1,1)-GRPO. These results indicate that a **limited exploration width m causes SAGE-RL to largely collapse to the standard GRPO optimization behavior**.

E.3. SAGE-RL shows Promising Potential in Difficult Reasoning Tasks

To more clearly elucidate the operational mechanism behind SAGE-RL, we compare the training dynamics of SAGE-GRPO-DS-1.5B (Ours) and GRPO-DS-1.5B (GRPO) on MATH-500 across five difficulty levels as training steps scales. The level 1-5 ranges from low to high, reflecting increasing levels of difficulty.

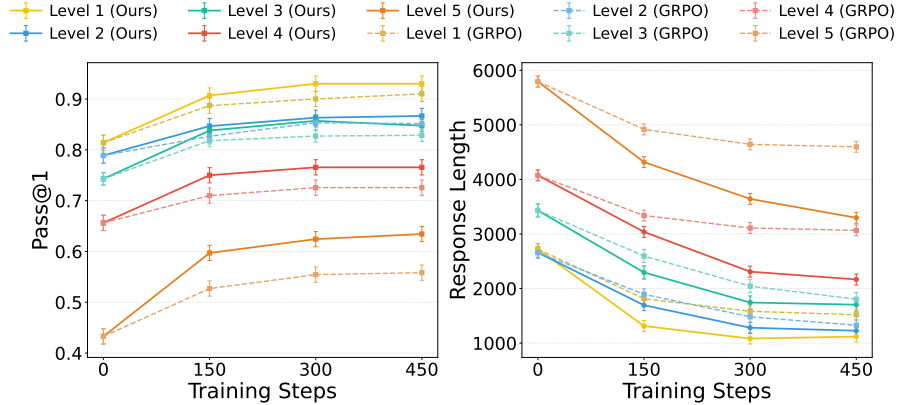


Figure 13. The training dynamics of SAGE-GRPO- DS-1.5B (Ours) and GRPO-DS-1.5B (GRPO) on MATH-500 across level 1-5.

As illustrated in Figure 13, both GRPO and SAGE-GRPO show steady performance gains across all difficulty levels of MATH500 as training progresses. SAGE-GRPO converges markedly faster than GRPO at every level and eventually attains performance comparable to GRPO on level 1-3 problems. **A clear divergence appears on level 4-5 problems, where SAGE-GRPO achieves substantially superior pass@1 and lower response length.** Remarkably, the downward trend in response length for SAGE-GRPO continues even after GRPO has converged.

These observations suggest that SAGE-RL primarily improves overall performance by dramatically increasing reasoning efficiency on difficult problems. This is consistent with the results in Table 7, which reveal significantly larger gains from SAGE-RL fine-tuning on more challenging benchmarks such as AIME 2024, AIME 2025, OlympiadBench, and Minerva than on relatively easier ones such as MATH-500 and AMC23.

Collectively, these findings highlight **the considerable potential of SAGE-RL in overcoming the reasoning performance bottlenecks faced by current LRMs on highly difficult tasks.**

E.4. Time Complexity Analysis

Time Complexity Analysis of SAGE SAGE generates $2m$ reasoning steps in parallel with a fixed exploration width m at each expansion step. Therefore, it theoretically achieves the same time complexity as Degrade SAGE, meanwhile, its space complexity is approximately $2m$ times higher. However, as we adopt vLLM (Kwon et al., 2023) as the inference engine, whose core design philosophy centers on a space-for-time tradeoff: it maximizes GPU memory utilization to minimize inference latency. Nevertheless, our implementation is constrained to the use of only 8 GPUs. Under this memory-limited setting, SAGE incurs higher inference-time cost compared to Degrade SAGE.

We report the average per-sample runtime of SAGE ($m, 1$) under different EW in this constrained hardware environment. Here, m denotes the exploration width EW. When $m = 0$, SAGE degenerates to Degrade SAGE. As shown in Figure 14(a), the inference time of DS-1.5B remains consistently higher than that of DeepScaleR. Moreover, the average inference time per response increases significantly with larger exploration widths.

This primarily arises from the trade-off adopted by vLLM: elevated space complexity is exchanged for reduced time complexity in the context of limited computational resources. In particular, once the exploration width exceeds 2, the growth rate of inference time accelerates further. Therefore, **we primarily set exploration width $m = 2$, which represents the transition**

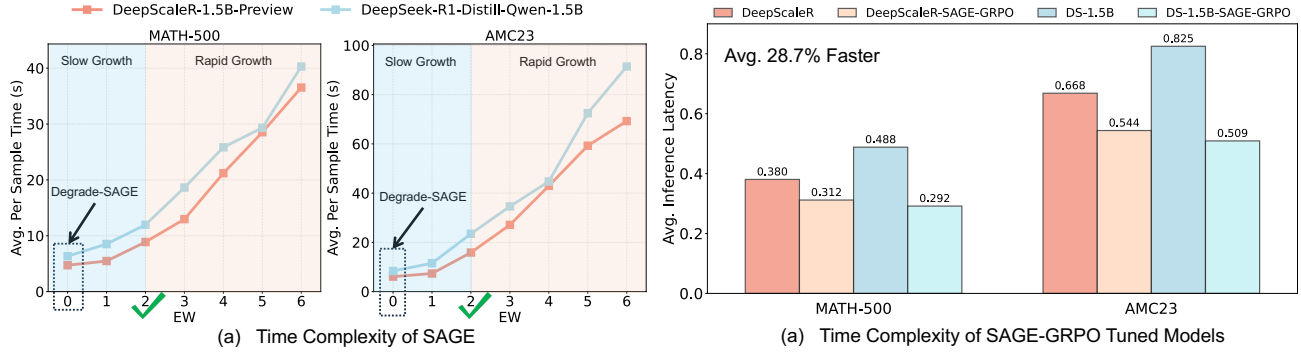


Figure 14. (a) Average inference time of SAGE on each question; (b) Comparison of Normalized inference time between the base models and the SAGE-GRPO tuned models on each question, approximated and normalized by the average response length.

point between the slow-growth and fast-growth regions, to achieve a balanced trade-off between efficiency and performance.

Time Complexity Analysis of SAGE-RL Tuned Models In the standard pass@1 inference setting, the KV cache is prefilled during the initial prompt processing phase, which ensures that the generation latency per subsequent token remains approximately constant. Consequently, for short queries, the total inference time of each completion scales nearly linearly with the number of generated tokens. However, vLLM aggressively optimizes inference speed through techniques such as KV cache reuse and continuous batching, which compromises the fairness of direct wall-clock time comparisons.

Given the approximately linear relationship between inference time and the number of generated tokens, we adopt the proxy metric $0.0001 \times (\text{average response length})$ to reflect the average inference latency. We compare this normalized metric between the base models and our SAGE-GRPO-tuned models. As shown in Figure 14(b), although SAGE incurs increasing inference-time cost with larger exploration widths under constrained hardware, SAGE-RL-tuned models can significantly reduce the average inference time in the standard pass@1 inference paradigm. Specifically, even on the relatively easier MATH-500 and AMC23 subsets among the six datasets we evaluated, our approach still achieves a 28.7% reduction in inference latency. When approximating average inference time using the average response length, Table 7 clearly shows that, compared to the baseline, **SAGE-RL-tuned models reduce inference latency by more than 40% across the majority of models and benchmarks.**

F. Hyperparameter Ablation

We conducted hyperparameter ablations for SAGE-GRPO and additional baselines beyond GRPO, including AdaptThink and ThinkPrune-2k. We swept the learning rate over $\{5 \times 10^{-7}, 2 \times 10^{-6}\}$ and the KL coefficient over $\{5 \times 10^{-4}, 2 \times 10^{-3}\}$, training each method independently under each setting.

Across all four configurations, the relative ranking is stable: SAGE-GRPO consistently achieves the highest Pass@1 and token efficiency. AdaptThink and ThinkPrune obtain shorter responses in some cases, but this compression comes at an accuracy cost; both underperform vanilla GRPO in Pass@1. These results indicate that the improvements of SAGE-GRPO are not artifacts of a particular hyperparameter choice.

G. Pseudo Code of SAGE

The example Python code of SAGE is illustrated in Figure 15, Figure 16 and Figure 17.

Table 9. Hyperparameter sweep over learning rate and KL coefficient on DS-1.5B. SAGE-GRPO consistently achieves the best Pass@1 and token efficiency on both AIME 2025 and OlympiadBench.

LR / KL	Method	AIME 2025			OlympiadBench		
		Pass@1	LEN	TE ($\times 10^{-3}$)	Pass@1	LEN	TE ($\times 10^{-3}$)
$5 \times 10^{-7} / 5 \times 10^{-4}$	SAGE-GRPO	25.8	7673	3.36	36.1	5253	6.87
$5 \times 10^{-7} / 5 \times 10^{-4}$	GRPO	23.3	8461	2.75	33.4	6536	5.11
$5 \times 10^{-7} / 5 \times 10^{-4}$	AdaptThink	21.9	8102	2.70	32.4	7306	6.85
$5 \times 10^{-7} / 5 \times 10^{-4}$	ThinkPrune	19.7	6950	2.83	32.3	4793	6.74
$5 \times 10^{-7} / 2 \times 10^{-3}$	SAGE-GRPO	25.5	7804	3.27	35.8	5387	6.65
$5 \times 10^{-7} / 2 \times 10^{-3}$	GRPO	23.0	8598	2.68	33.1	6665	4.97
$5 \times 10^{-7} / 2 \times 10^{-3}$	AdaptThink	22.0	8254	2.67	31.5	7626	6.61
$5 \times 10^{-7} / 2 \times 10^{-3}$	ThinkPrune	19.9	6925	2.87	31.6	4779	6.61
$2 \times 10^{-6} / 5 \times 10^{-4}$	SAGE-GRPO	26.0	7644	3.40	36.3	5223	6.95
$2 \times 10^{-6} / 5 \times 10^{-4}$	GRPO	23.5	8436	2.79	33.6	6491	5.18
$2 \times 10^{-6} / 5 \times 10^{-4}$	AdaptThink	22.0	8202	2.68	32.7	4720	6.93
$2 \times 10^{-6} / 5 \times 10^{-4}$	ThinkPrune	19.8	6934	2.86	32.5	4790	6.78
$2 \times 10^{-6} / 2 \times 10^{-3}$	SAGE-GRPO	26.2	7608	3.44	36.6	5177	7.07
$2 \times 10^{-6} / 2 \times 10^{-3}$	GRPO	23.8	8399	2.83	33.9	6452	5.25
$2 \times 10^{-6} / 2 \times 10^{-3}$	AdaptThink	21.8	8250	2.64	33.0	4743	6.96
$2 \times 10^{-6} / 2 \times 10^{-3}$	ThinkPrune	19.9	6921	2.88	32.7	4802	6.81

```

# Initialization and Parameter Processing

class SAGE:
    def __init__(self, llm, tokenizer, eostep_token='\n\n'):
        self.llm = llm
        self.tokenizer = tokenizer
        self._init_special_tokens(eostep_token)

    def forward(self, prompts, params):

        # Process and validate input parameters
        exploration_params = self._process_params(params)

        # Initialize exploration instances for each prompt
        instances = self._init_exploration_instances(prompts, exploration_params)

        # Perform step-wise exploration reasoning
        self._stepwise_exploration(instances, exploration_params)

        # Generate final answers from completed reasoning
        return self._generate_final_answers(instances, exploration_params)

    def _init_special_tokens(self, eostep_token):
        self.eot_token_id = self.tokenizer.convert_tokens_to_ids('</think>')
        self.sot_token_id = self.tokenizer.convert_tokens_to_ids('<think>')
        self.eostep_token_id = self.tokenizer.convert_tokens_to_ids(eostep_token)
        self.eostep_token_list = [self.eostep_token_id, self.eot_token_id]

    def _process_params(self, params):
        processed = {}
        processed['exploration_width'] = params.get('exploration_width', 4)
        processed['max_ans_tokens'] = params.get('max_ans_tokens', 100)
        processed['max_step_num'] = params.get('max_step_num', 10)
        processed['per_step_token'] = params.get('per_step_token', 100)
        processed['enable_sort'] = params.get('enable_sort', True)
        processed['required_completions'] = params.get('required_completions', 1)
        processed['insert_sot'] = params.get('insert_sot', True)
        processed['top_p'] = params.get('top_p', 1.0)
        processed['temperature'] = params.get('temperature', 1.0)
        return processed

    def _init_exploration_instances(self, prompts, params):
        instances = []
        for prompt in prompts:
            # Convert prompt to tokens if it's a string
            prompt_tokens = self.tokenizer.encode(prompt) if isinstance(prompt, str) else prompt

            # Add start-of-thinking token if requested
            if params['insert_sot']:
                prompt_tokens.append(self.sot_token_id)

            # Create exploration instance with empty candidates
            instance = ExplorationInstance(
                prompt_tokens,
                logprobs=None,
                required_completions=params['required_completions']
            )
            instances.append(instance)
        return instances

```

Figure 15. Example Python code illustrating the SAGE Initialization and Parameter Processing.

```

# Step-wise Exploration

def _stepwise_exploration(self, instances, params):
    for t in range(params['max_step_num']):
        active_instances = [inst for inst in instances if not inst.is_finished]
        if not active_instances:
            break

        for instance in active_instances:
            # Initialize candidates if empty
            if not instance.candidates:
                self._init_instance_candidates(instance)

            # Generate next reasoning steps
            new_candidates = self._generate_next_steps(instance, params)

            # Process and update candidates
            self._update_instance_candidates(instance, new_candidates, params)

            # Check if instance has enough completions
            if len(instance.completed) >= instance.required_completions:
                instance.is_finished = True
                instance.candidates = []

def _init_instance_candidates(self, instance):
    instance.candidates.append(ExplorationSequence(
        tokens=instance.prompt_tokens,
        logprobs=[],
        cum_logprob=0.0))

def _generate_next_steps(self, instance, params):
    # Create batch of prompts from current candidates
    candidate_prompts = [cand.tokens for cand in instance.candidates]

    # Configure step sampling parameters
    step_params = SamplingParams(
        n=params['exploration_width'] * 2,
        temperature=params['temperature'],
        top_p=params['top_p'],
        max_tokens=params['per_step_token'],
        stop_token_ids=self.eostep_token_list)

    # Generate next steps using LLM
    outputs = self.llm.generate(
        prompt_token_ids=candidate_prompts,
        sampling_params=step_params,
        use_tqdm=False)

    # Process outputs into new candidates
    return self._process_step_outputs(instance.candidates, outputs)

def _process_step_outputs(self, current_candidates, outputs):
    new_candidates = []
    for i, (candidate, output) in enumerate(zip(current_candidates, outputs)):
        if output and output.outputs:
            for step_output in output.outputs:
                new_candidate = ExplorationSequence(
                    tokens=candidate.tokens + step_output.token_ids,
                    logprobs=candidate.logprobs + step_output.logprobs,
                    cum_logprob=candidate.cum_logprob + step_output.cumulative_logprob)
                new_candidates.append(new_candidate)
    return new_candidates

```

Figure 16. Example Python code illustrating the Step-wise Exploration of SAGE.

```

# Candidate Management and Scoring

def _update_instance_candidates(self, instance, new_candidates, params):
    completed, ongoing = self._separate_completed_candidates(new_candidates)
    instance.completed.extend(completed)
    if params['enable_sort']:
        ongoing.sort(key=self._score_candidate, reverse=True)
    instance.candidates = ongoing[:params['exploration_width']]

def _separate_completed_candidates(self, candidates):
    completed = []
    ongoing = []
    for candidate in candidates:
        if candidate.tokens[-1] == self.eot_token_id:
            completed.append(candidate)
        else:
            ongoing.append(candidate)
    return completed, ongoing

def _score_candidate(self, candidate):
    return candidate.cum_logprob / len(candidate.tokens)

# Final Answer Generation

def _generate_final_answers(self, instances, params):
    outputs = []
    for instance in instances:
        # Combine and sort all candidates
        all_candidates = instance.completed + instance.candidates
        all_candidates.sort(key=self._score_candidate, reverse=True)

        # Get best candidates and ensure proper formatting
        best_candidates = all_candidates[:instance.required_completions]
        self._finalize_candidates(best_candidates)

        # Generate answers based on completed reasoning
        answer_candidates = self._generate_answers_from_candidates(best_candidates, params)

        # Create output structure
        outputs.append(ExplorationOutput(sequences=answer_candidates))
    return outputs

def _finalize_candidates(self, candidates):
    for candidate in candidates:
        if self.eot_token_id not in candidate.tokens:
            candidate.tokens.append(self.eot_token_id)

def _generate_answers_from_candidates(self, candidates, params):
    candidate_prompts = [cand.tokens for cand in candidates]
    answer_params = SamplingParams(
        n=1, temperature=0.0,
        max_tokens=params['max_ans_tokens'])
    answer_outputs = self.llm.generate(
        prompt_token_ids=candidate_prompts,
        sampling_params=answer_params, use_tqdm=False)
    for candidate, output in zip(candidates, answer_outputs):
        if output and output.outputs:
            candidate.tokens += output.outputs[0].token_ids
            candidate.text = self.tokenizer.decode(candidate.tokens)
    return candidates

```

Figure 17. Example Python code illustrating the Candidate Management, Scoring and Final Answer Generation of SAGE.

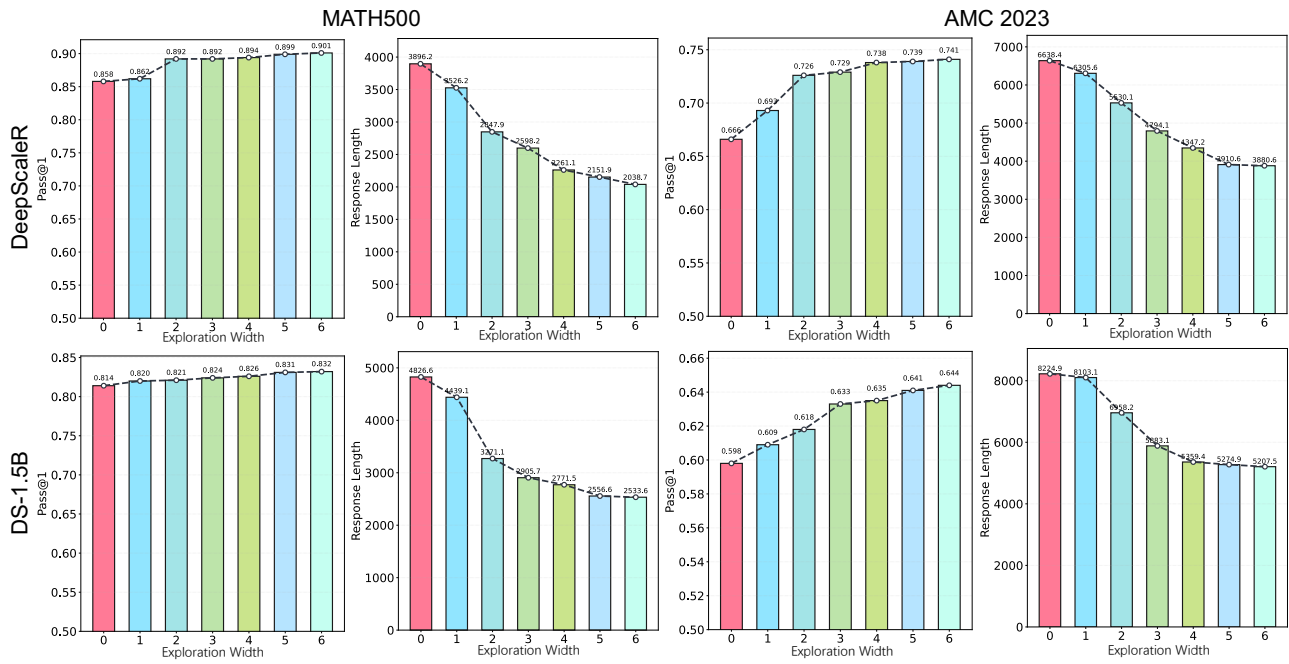


Figure 18. Performance on DeepScaleR and DS-1.5B with different exploration width on MATH500 and AMC23. Under all settings, both pass@1 and response length gradually converge.

DS-1.5B	SAGE-GRPO-DS-1.5B
Question Correct Answer: 800 John buys twice as many red ties as blue ties. The red ties cost 50% more than blue ties. He spent \$200 on blue ties that cost \$40 each. How much did he spend on ties?	Question Correct Answer: 800 John buys twice as many red ties as blue ties. The red ties cost 50% more than blue ties. He spent \$200 on blue ties that cost \$40 each. How much did he spend on ties?
Reasoning 957 tokens Okay, so I need to figure out how much John spent on ties. Let's start by parsing the information given step by step. First, John buys twice as many red ties as blue ties. Hmm, that means if he buys x blue ties, he buys $2x$ red ties. That makes sense. Next, the red ties cost 50% more than the blue ties. Alright, so if the blue tie is priced at, say, y dollars, then red ties would cost y plus 50% of y, which is $1.5y$. Alternatively, it can be written as $1.5y$. ... Now, let's figure out the cost per red tie. We know that the red ties cost 50% more than the blue ties. The blue tie costs \$40, so 50% of 40 is 20. Therefore, each red tie costs $40 + 20 = \\$60$. Total spent = Total cost of blue ties + Total cost of red ties Total spent = $200 + 600 = \\$800$. So, John spent a total of \$800 on ties. Wait, let me double-check my calculations to be sure I didn't make any mistakes. First, blue ties: $200 / 40$ is indeed 5 ties. Red ties are twice that, so 10 ties. Each blue tie is \$40, red tie is 50% more, which is \$60. 10 red ties at \$60 each: $10 * 60$ is 600. Blue ties total \$200. So $600 + 200$ is 800. Yeah, that checks out. ... Total expenditure: $200 + 600 = 800$. Yeah, definitely \$800. I think that even if I messed up in the calculation, this straightforward approach seems correct. So, he spent \$800 on ties. **Final Answer** John spent a total of $\boxed{\\$800}$ on ties.	Reasoning 467 tokens Okay, so I need to figure out how much John spent on ties. Let's start by parsing the information given step by step. First, John buys twice as many red ties as blue ties, if he buys x blue ties, he buys $2x$ red ties. Red ties are 50% more expensive than blue ties. For a blue tie priced at y dollars, the red tie price is $1.5y$. ... Now, let's figure out the cost per red tie. Since red ties are 50% more expensive than blue ties (\$40), each red tie costs $40 + 20 = \\$60$. Total spent = Total cost of blue ties + Total cost of red ties Total spent = $200 + 600 = \\$800$. So, John spent a total of \$800 on ties. **Final Answer** John spent a total of $\boxed{\\$800}$ on ties.

Figure 19. Case Study 1

DS-1.5B	SAGE-GRPO-DS-1.5B
Question Correct Answer: $(3, \frac{\pi}{2})$ Convert the point $(0,3)$ in rectangular coordinates to polar coordinates.	Question Correct Answer: $(3, \frac{\pi}{2})$ Convert the point $(0,3)$ in rectangular coordinates to polar coordinates.
Reasoning 1381 tokens Okay, so I need to convert the rectangular coordinate $(0, 3)$ to polar coordinates. Hmm, I remember that polar coordinates are represented as (r, θ)... First, let me recall the formulas ... I think the formula for r ... But wait, I also remember that sometimes $\arctan(y/x)$ doesn't give the correct angle because it depends on the quadrant where the point is located. So, I need to be careful about that. Alright, let's apply this to the point $(0, 3)$... Okay, so r is 3. That makes sense because the point is 3 units above the origin on the y-axis, so the distance from the origin is 3. ... Now, let's figure out θ. Since x is 0, we have $\theta = \arctan(3/0)$. Wait, division by zero is undefined. Hmm, that's a problem. I need to think about what angle θ would make the point $(0, 3)$ in polar coordinates. I know that when x is 0, the point lies on the y-axis. If y is positive, it's on the positive y-axis, which is at 90 degrees or $\pi/2$ radians from the positive x-axis. If y were negative, it would be at 270 degrees or $3\pi/2$ radians. Since our y is positive, θ should be $\pi/2$. Wait, but sometimes when x is 0, the angle is either $\pi/2$ or $3\pi/2$, but since our y is positive, it's $\pi/2$. So, θ is $\pi/2$. So, putting it all together, the polar coordinates are $(3, \pi/2)$. Let me just double-check. ... $\cos(\pi/2)$ is 0, so x is 0. y is $r \sin \theta = 3 \sin(\pi/2)$. $\sin(\pi/2)$ is 1, so y is 3. That gives us $(0, 3)$, which is exactly the point we started with. So, that seems correct. ... So, putting it all together, the polar coordinates are $(3, \pi/2)$. Let me just double-check ... That gives us $(0, 3)$, which is exactly the point we started with. So, that seems correct. **Final Answer** The polar coordinates are: $\boxed{(3, \frac{\pi}{2})}$	Reasoning 712 tokens Okay, so I need to convert the rectangular coordinate $(0, 3)$ to polar coordinates. Hmm, I remember that polar coordinates are represented as (r, θ)... First, let me recall the formulas for converting rectangular coordinates (x, y) to polar coordinates (r, θ). I think the formula for r is the square root of $(x^2 + y^2)$. So, $r = \sqrt{x^2 + y^2}$. And for θ, I believe it's the arctangent of y over x, so $\theta = \arctan(y/x)$. Alright, let's apply this to the point $(0, 3)$... Okay, so r is 3. That makes sense because the point is 3 units above the origin on the y-axis, so the distance from the origin is 3. ... I remember that angles in polar coordinates are measured from the positive x-axis, going counterclockwise ... Let me confirm that. Yes, $\pi/2$ radians is 90 degrees, which points straight up along the y-axis. So, θ should be $\pi/2$. So, putting it all together, the polar coordinates should be $(3, \pi/2)$. Let me just write that down: $(3, \pi/2)$. **Final Answer** The polar coordinates are: $\boxed{(3, \frac{\pi}{2})}$

Figure 20. Case Study 2